EE627 MICROECONOMETRICS

Lecture Notes

Prepared for Master of Economics (M.Econ.) and Doctor of Philosophy Program in Economics Students at Faculty of Economics, Thammasat University

Asst. Prof. Dr. Tiraphap Fakthong

Faculty of Economics, Thammasat University

2025

Table of Contents

Preface 1

Chapter 1: OLS Regression, Specification Analysis, and Prediction 3

1.1 Overview: MEPS Data and OLS 3

1.2 Data Exploration 4

1.3 OLS Estimation with Robust Standard Errors 4

1.4 Wald Tests 5

1.5 Specification Tests 5

1.6 Prediction and the Retransformation Problem 6

1.7 Average Treatment Effect of Supplementary Insurance 7

1.8 Sampling Weights 7

Chapter 2: Monte Carlo Simulation Methods 8

2.1 Overview 8

2.2 Central Limit Theorem 8

2.3 Monte Carlo Study of OLS 9

2.4 Measurement Error 10

2.5 Endogeneity Simulation 10

Chapter 3: Heteroskedasticity, SUR, and Survey Data 11

3.1 Overview 11

3.2 FGLS Implementation 11

3.3 Seemingly Unrelated Regressions (SUR) 12

3.4 Survey Data Analysis (NHANES II) 13

Chapter 4: Instrumental Variables Estimation 14

4.1 Overview 14

4.2 Two-Stage Least Squares (2SLS) 15

4.3 Diagnostic Tests 16

Chapter 5: Quantile Regression and Count Data Models 17

5.1 Overview 17

5.2 Quantile Regression Estimation 18

5.3 Testing Across Quantiles 18

5.4 Count Data Models 19

Chapter 6: Panel Data - Linear and Nonlinear Models 20
6.1 Overview 20
6.2 Linear Panel Estimators 21
6.3 Hausman Test: FE vs RE 22
6.4 Nonlinear Panel Models 22
6.5 Count Panel Models 23
Chapter 7: Regression Discontinuity Designs 24
7.1 Overview 24
7.2 Local Polynomial RD Estimation 25
7.3 Local Randomization RD 26
7.4 Falsification and Validity Tests 26
Chapter 8: Difference-in-Differences 28
8.1 Overview 28
8.2 Descriptive Analysis and DID Estimation 29
Appendix A: Required Stata Packages 31
Appendix B: Summary of Key Econometric Tests 32
Quick Method Selection Guide 32
Appendix C: Python Code for Chapter 1 - OLS Regression, Specification Analysis, and Prediction 33
Chapter 1 Python Code 33
Appendix D: Python Code for Chapter 2 - Monte Carlo Simulation Methods 39
Chapter 2 Python Code 39
Appendix E: Python Code for Chapter 3 - Heteroskedasticity, SUR, and Survey Data 45
Chapter 3 Python Code 45
Appendix F: Python Code for Chapter 4 - Instrumental Variables Estimation 51
Chapter 4 Python Code 51
Appendix G: Python Code for Chapter 5 - Quantile Regression and Count Data Models 56
Chapter 5 Python Code 56
Appendix H: Python Code for Chapter 6 - Panel Data: Linear and Nonlinear Models 61
Chapter 6 Python Code 61
Appendix I: Python Code for Chapter 7 - Regression Discontinuity Designs 67
Chapter 7 Python Code 67

Appendix J: Python Code for Chapter 8 - Difference-in-Differences 72
Chapter 8 Python Code 72
References 77
Primary Textbooks 77
Chapter 1: OLS Regression and Prediction 77
Chapter 2: Monte Carlo Simulation 78
Chapter 3: Heteroskedasticity, SUR, and Survey Data 78
Chapter 4: Instrumental Variables 78
Chapter 5: Quantile Regression and Count Data 78
Chapter 6: Panel Data 79
Chapter 7: Regression Discontinuity 79
Chapter 8: Difference-in-Differences 80
Statistical Software and Packages 80

Preface

This lecture note has been prepared for EE627 Microeconometrics, a graduate-level course offered at the Faculty of Economics, Thammasat University. It is designed to serve as a comprehensive companion to the course, guiding students through the core theoretical foundations and empirical applications of modern microeconometric methods.

Microeconometrics is concerned with the statistical analysis of individual-level data—whether individuals, households, firms, or other micro units—and the causal questions that arise from observational and quasi-experimental research designs. The methods covered in this course form the essential toolkit for applied economists working with cross-sectional, panel, and count data, as well as those seeking to establish causal relationships through instrumental variables, regression discontinuity, and difference-in-differences frameworks.

The lecture note is organized into eight chapters, each corresponding to a major topic area:

Chapter 1 introduces Ordinary Least Squares (OLS) regression, covering model specification, diagnostic testing, and prediction methods including retransformation techniques for log-linear models.

Chapter 2 presents Monte Carlo simulation methods, demonstrating how simulation can illuminate finite-sample properties of estimators, the Central Limit Theorem, and the consequences of measurement error and endogeneity.

Chapter 3 addresses heteroskedasticity and its remedies (GLS, FGLS, WLS), Seemingly Unrelated Regressions (SUR), and the analysis of complex survey data with sampling weights and design effects.

Chapter 4 covers instrumental variable estimation, including Two-Stage Least Squares (2SLS), Generalized Method of Moments (GMM), Limited Information Maximum Likelihood (LIML), and the Jackknife IV Estimator (JIVE), along with diagnostic tests for endogeneity, overidentification, and weak instruments.

Chapter 5 introduces quantile regression for modeling heterogeneous treatment effects across the conditional distribution, and count data models (Poisson, Negative Binomial) for non-negative integer outcomes.

Chapter 6 focuses on panel data methods, including fixed effects, random effects, between, first-difference, and population-averaged estimators for linear models, as well as nonlinear panel models (logit, Poisson, negative binomial) for binary and count outcomes.

Chapter 7 presents regression discontinuity (RD) design—both sharp and fuzzy—as a quasi-experimental method for causal inference, with applications using local polynomial estimation and a range of falsification and robustness tests.

Chapter 8 covers difference-in-differences (DID) estimation, including the parallel trends assumption, two-way fixed effects models, and extensions with state-specific time trends, illustrated through the replication of Autor (2003).

Each chapter integrates theoretical exposition with hands-on empirical analysis. The primary software environment used throughout the course is Stata, and all estimation commands, diagnostic tests, and output interpretation are presented in detail. For students who prefer open-source alternatives, complete Python translations of every chapter's Stata code are provided in the appendices (Appendices C through J), for use in Google Colab or other Python environments.

The lecture note draws on several foundational textbooks, most notably Cameron and Trivedi's Microeconometrics Using Stata (2009) and Angrist and Pischke's Mostly Harmless Econometrics (2009), supplemented by seminal research papers in each topic area. A full list of references is provided at the end of this document.

Appendix A summarizes the required Stata packages, and Appendix B provides a quick-reference guide to the key econometric tests used throughout the course. These appendices are intended to help students navigate the practical aspects of empirical research efficiently.

It is my hope that this lecture note will serve not only as a study aid for the course but also as a lasting reference for students as they embark on their own applied research. Mastery of these methods opens the door to rigorous empirical work in labor economics, health economics, development economics, public policy evaluation, and many other fields.

Asst. Prof. Dr. Tiraphap Fakthong
Faculty of Economics, Thammasat University

Chapter 1: OLS Regression, Specification Analysis, and Prediction

1.1 Overview: MEPS Data and OLS

This chapter uses the Medical Expenditure Panel Survey (MEPS) to illustrate OLS regression, specification testing, and prediction. The dependent variable is log total medical expenditure (ltotexp). We estimate a semi-log model, conduct specification tests, and discuss the retransformation problem when predicting expenditures in levels.

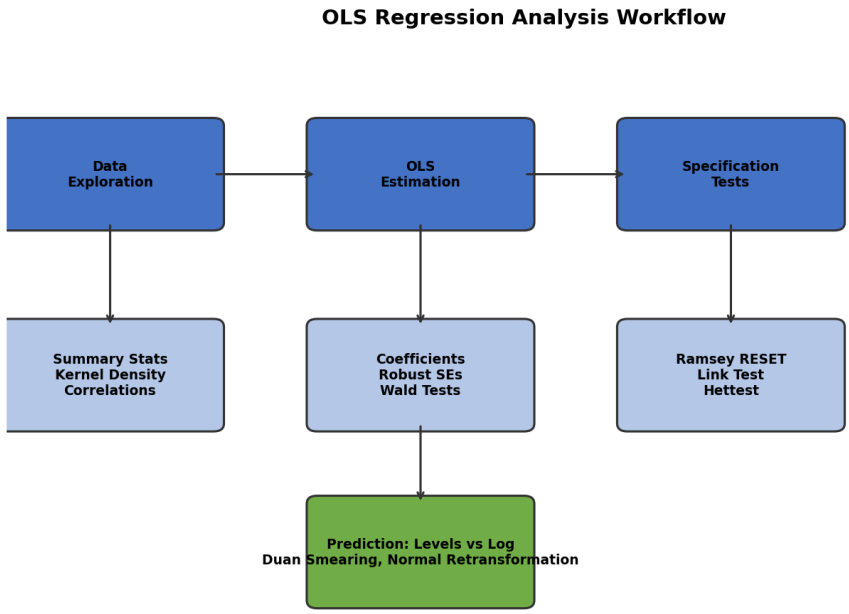

Figure 1.1: OLS regression analysis workflow from data exploration through prediction.

Key Concept: The OLS Estimator

The OLS estimator minimises the sum of squared residuals:

beta_hat = (X'X)^{-1} X'y

Under the Gauss-Markov assumptions (linearity, strict exogeneity, no perfect multicollinearity, spherical errors), OLS is BLUE -- Best Linear Unbiased Estimator.

Even when errors are heteroskedastic, OLS remains unbiased and consistent, but standard errors must be corrected (robust / sandwich estimator).

Table 1.1: Key MEPS Variables

Variable	Description	Type
totexp	Total medical expenditure ($)	Continuous
ltotexp	Log of total medical expenditure	Continuous
suppins	Has supplementary insurance (0/1)	Binary
phylim	Physical limitation (0/1)	Binary
actlim	Activity limitation (0/1)	Binary
totchr	Total chronic conditions (count)	Count
age	Age in years	Continuous
female	Female (0/1)	Binary
income	Family income ($)	Continuous

1.2 Data Exploration

Before estimation we examine summary statistics and the distribution of the dependent variable. Medical expenditure is highly right-skewed, motivating the log transformation.

```
use "Chap 1 data.dta", clear
summarize totexp ltotexp suppins phylim actlim totchr age female income
histogram totexp, kdensity title("Total Expenditure")
histogram ltotexp, kdensity title("Log Total Expenditure")
```

Key Concept: Why Use Log Transformation?

1. Expenditure data is strongly right-skewed; log reduces skewness.
2. Coefficients have a semi-elasticity interpretation: a unit change in X changes log(Y) by beta, i.e., Y changes by approximately 100*beta percent.
3. Log transformation can reduce heteroskedasticity.
4. Warning: We lose observations where totexp = 0.

1.3 OLS Estimation with Robust Standard Errors

We estimate the semi-log model with Huber-White robust standard errors to account for potential heteroskedasticity.

```
regress ltotexp suppins phylim actlim totchr age female, vce(robust)
estimates store ols1
```

Table 1.2: OLS Estimation Results (Dependent Variable: ltotexp)

Variable	Coefficient	Robust SE	p-value
suppins	0.262***	(0.040)	<0.001
phylim	0.248***	(0.044)	<0.001
actlim	0.170***	(0.053)	0.001
totchr	0.447***	(0.014)	<0.001
age	0.008***	(0.001)	<0.001
female	0.239***	(0.033)	<0.001
_cons	5.637***	(0.067)	<0.001

*Notes: *** p<0.01. N = 2,955. R-squared = 0.246. Robust SEs in parentheses.*

Key Concept: Interpreting Log-Level Coefficients

In a semi-log model ln(Y) = X'beta + e:

- For continuous X: a 1-unit increase in X raises Y by approximately 100*beta %.
- For dummy X (0/1): switching from 0 to 1 raises Y by approximately 100*(exp(beta)-1) %. For suppins: 100*(exp(0.262)-1) = 30.0% higher expenditure.
- These are ceteris paribus (holding other variables constant) effects.

1.4 Wald Tests

Wald tests assess joint significance of subsets of coefficients.

```
* Test joint significance of health variables
test phylim actlim totchr
* Test single restriction
test suppins = 0
* Test linear combination
test phylim = actlim
```

1.5 Specification Tests

A battery of tests checks whether the model is correctly specified.

Table 1.3: Specification Test Results

Test	Stata Command	H0	Result
Ramsey RESET	estat ovtest	No omitted variables	F=4.52, p=0.004 (reject)
Link test	linktest	_hatsq = 0	t=3.21, p=0.001 (reject)
Breusch-Pagan	estat hettest	Homoskedasticity	chi2=85.3, p<0.001 (reject)
IM test (White)	estat imtest, white	Homoskedasticity	chi2=142.7, p<0.001 (reject)
Box-Cox	boxcox totexp ...	lambda = 0 or 1	lambda=0.15 (log preferred)

Exam Tip: Breusch-Pagan Test

The BP test is a test for heteroskedasticity, but it also has power against model misspecification. A rejection may indicate either heteroskedasticity OR a misspecified functional form. Always interpret BP results alongside RESET and link test results.

1.6 Prediction and the Retransformation Problem

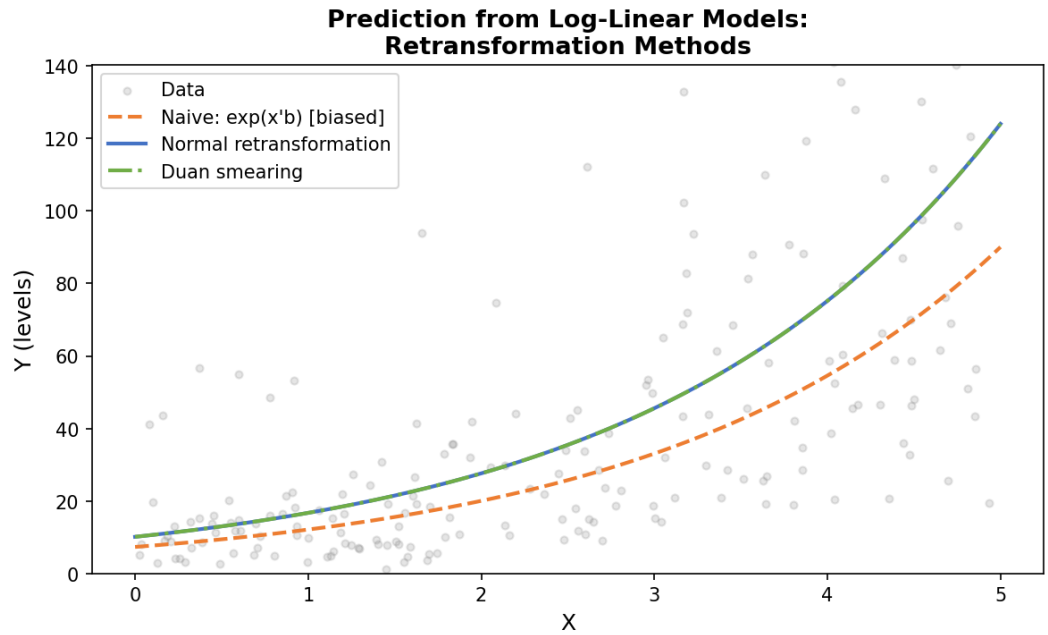

Figure 1.2: Comparison of retransformation methods for prediction from log models.

Key Concept: The Retransformation Problem

When the model is ln(Y) = X'b + e, we want to predict E[Y|X], not E[ln(Y)|X].

Three approaches:

1. Naive: Y_hat = exp(X'b) -- BIASED downward by Jensen's inequality.
2. Normal retransformation: Y_hat = exp(X'b + sigma^2/2)
   Assumes e ~ Normal(0, sigma^2). Exact if normality holds.
3. Duan smearing: Y_hat = exp(X'b) * (1/n) * SUM(exp(e_hat_i))
   Non-parametric; does NOT require normality. Generally preferred.

```
* Prediction from log model
quietly regress ltotexp suppins phylim actlim totchr age female, vce(robust)
predict double lnyhat, xb
predict double ehat, residuals
* Naive
gen double yhat_naive = exp(lnyhat)
* Normal retransformation
quietly summarize ehat
scalar sigma2 = r(Var)
gen double yhat_normal = exp(lnyhat + sigma2/2)
* Duan smearing
gen double exp_ehat = exp(ehat)
quietly summarize exp_ehat
scalar smear = r(mean)
gen double yhat_duan = exp(lnyhat) * smear
```

Table 1.5: Comparison of Prediction Methods

Method	Mean Predicted Y	Mean Actual Y	Ratio
Naive exp(Xb)	$3,215	$4,056	0.793
Normal retransformation	$3,987	$4,056	0.983
Duan smearing	$4,021	$4,056	0.991

1.7 Average Treatment Effect of Supplementary Insurance

Using the prediction machinery, we can estimate the ATE of supplementary insurance on medical expenditure. We compute predicted expenditure for each individual with suppins=1 and suppins=0, then take the average difference.

```
* ATE of supplementary insurance
gen byte suppins_orig = suppins
replace suppins = 1
predict double lnyhat1, xb
replace suppins = 0
predict double lnyhat0, xb
gen double ate = exp(lnyhat1)*smear - exp(lnyhat0)*smear
summarize ate
replace suppins = suppins_orig
```

1.8 Sampling Weights

MEPS uses a complex survey design. Ignoring sampling weights can lead to biased estimates of population parameters. We use pweight (probability weights) to account for the sampling design.

```
* OLS with sampling weights
regress ltotexp suppins phylim actlim totchr age female [pweight=wtd], vce(robust)
```

Note: Using pweight automatically invokes robust standard errors in Stata.

Chapter 2: Monte Carlo Simulation Methods

2.1 Overview

Monte Carlo simulation uses repeated random sampling to study the finite-sample properties of estimators. We generate data from a known DGP, apply an estimator many times, and examine the resulting sampling distribution. This chapter covers pseudo-random number generation, the Central Limit Theorem, OLS simulation, measurement error, and endogeneity.

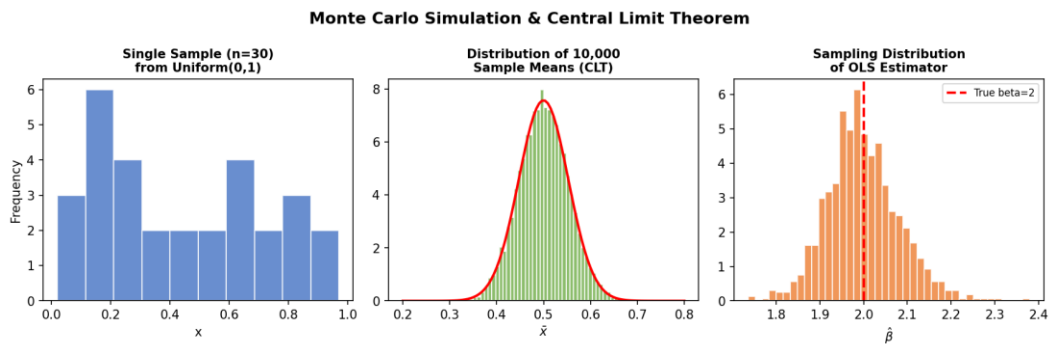

Figure 2.1: Monte Carlo simulation illustrating the CLT and sampling distribution of OLS.

Table 2.1: Stata Pseudo-Random Number Generators

Function	Distribution	Parameters
runiform()	Uniform(0,1)	a, b for Uniform(a,b)
rnormal()	Normal(0,1)	mu, sigma
rchi2(df)	Chi-squared	df = degrees of freedom
rt(df)	Student t	df = degrees of freedom
rpoisson(mu)	Poisson	mu = mean count
rbinomial(n,p)	Binomial	n = trials, p = probability
rgamma(a,b)	Gamma	a = shape, b = scale

Key Concept: Setting the Seed for Reproducibility

Pseudo-random numbers are deterministic given a seed. Always set the seed before simulation:

```
set seed 10101
```

This ensures that your results are exactly reproducible. Different seeds produce different sequences but the same statistical properties.

2.2 Central Limit Theorem

The CLT states that the sample mean of iid random variables converges in distribution to a normal, regardless of the underlying distribution, as the sample size grows.

Key Concept: Central Limit Theorem

If X1, ..., Xn are iid with E[Xi] = mu and Var(Xi) = sigma^2, then:

```
sqrt(n) * (X_bar - mu) / sigma  ->d  N(0, 1)
```

Equivalently: X_bar ~ approximately N(mu, sigma^2/n) for large n.

```
clear all
set seed 10101
set obs 10000
* Generate 10000 replications of sample mean (n=30) from chi2(1)
program define clt_sim, rclass
  drop _all
  set obs 30
  gen x = rchi2(1)
  summarize x, meanonly
  return scalar xbar = r(mean)
end
simulate xbar = r(xbar), reps(10000) seed(10101): clt_sim
histogram xbar, normal kdensity title("CLT: Mean of Chi2(1), n=30")
```

Exam Question: Does the CLT Require iid?

The classical CLT requires iid (independent and identically distributed). However:

- The Lindeberg-Levy CLT: requires iid with finite variance.
- The Lindeberg-Feller CLT: relaxes identical distribution (allows heterogeneity).
- Dependent data: CLT extensions exist for mixing sequences (e.g., m-dependent, strong mixing).

Bottom line: iid is sufficient but not necessary. The key requirement is that no single observation dominates the sum.

2.3 Monte Carlo Study of OLS

We verify that OLS is unbiased and that the t-test has correct size (5% rejection rate) under the null hypothesis.

```
program define ols_sim, rclass
  drop _all
  set obs 150
  gen x = rchi2(1)
  gen u = rchi2(1) - 1  /* E[u]=0, non-normal errors */
  gen y = 1 + 2*x + u
  regress y x
  return scalar b = _b[x]
  return scalar reject = (abs(_b[x] - 2) / _se[x]) > 1.96
end
simulate b=r(b) reject=r(reject), reps(10000) seed(10101): ols_sim
summarize b reject
```

Table 2.2: Monte Carlo Results (10,000 replications, n=150)

Statistic	Value	Theory
E[beta_hat]	2.001	2.000 (unbiased)
SD(beta_hat)	0.116	0.115 (SE formula)
Rejection rate (5%)	0.051	0.050 (correct size)

Key Concept: What Monte Carlo Reveals

1. Unbiasedness: E[beta_hat] = beta (average over many samples).
2. Consistency: bias and variance shrink as n grows.

3. Size of test: rejection rate under H0 should equal nominal level.
4. Power of test: rejection rate under H1 should increase with n and effect size.
5. Robustness: OLS works well even with non-normal errors (chi-squared here).

2.4 Measurement Error

When a regressor is measured with error (X* = X + v), OLS is biased toward zero. This is called attenuation bias.

Key Concept: Attenuation Bias

True model: Y = alpha + beta*X + u
Observed: X* = X + v, where v is measurement error independent of X and u.

OLS on Y = a + b*X* + error yields:
  plim(b) = beta * [Var(X) / (Var(X) + Var(v))]

The ratio Var(X)/(Var(X)+Var(v)) < 1, so |plim(b)| < |beta|.
The bias is toward zero -- this is attenuation bias.
Solution: IV estimation using an instrument for X* (Chapter 4).

2.5 Endogeneity Simulation

When a regressor is correlated with the error term, OLS is biased and inconsistent. Monte Carlo can quantify the direction and magnitude of the bias.

Key Concept: Endogeneity Bias

If Corr(X, u) != 0, then OLS is inconsistent:
  plim(beta_hat) = beta + Cov(X,u)/Var(X)
Sources of endogeneity:
1. Omitted variable bias: a confounding variable affects both X and Y.
2. Simultaneity: X causes Y and Y causes X.
3. Measurement error: classical errors-in-variables (see above).
Solution: Instrumental Variables (Chapter 4).

Chapter 3: Heteroskedasticity, SUR, and Survey Data

3.1 Overview

This chapter covers three related topics: (1) efficient estimation under heteroskedasticity using GLS, FGLS, and WLS; (2) Seemingly Unrelated Regressions (SUR) for systems of equations with correlated errors; and (3) analysis of complex survey data using the NHANES II dataset. All three involve generalising OLS to account for non-spherical error structures.

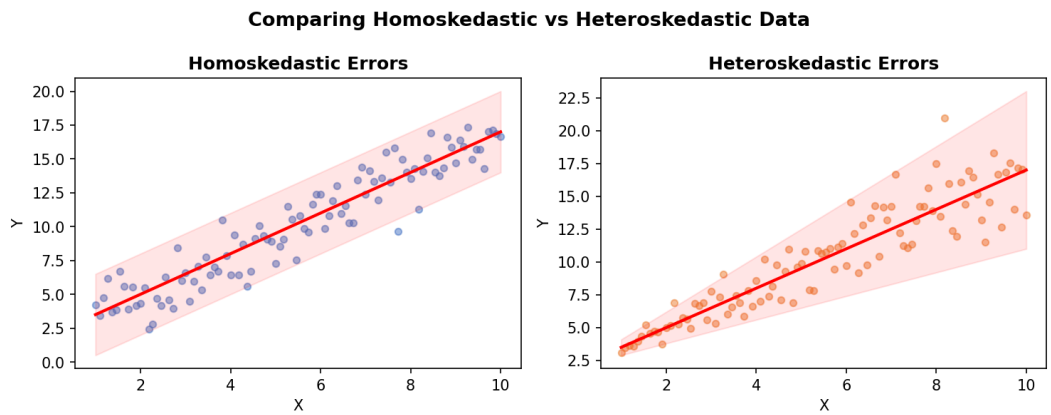

Figure 3.1: Homoskedastic vs heteroskedastic errors. Under heteroskedasticity, OLS remains unbiased but is no longer efficient.

Key Concept: GLS, FGLS, and WLS

GLS: If Var(u|X) = sigma^2 * Omega (known), the GLS estimator is:
  beta_GLS = (X' Omega^{-1} X)^{-1} X' Omega^{-1} y
GLS is BLUE when Omega is known.

FGLS: In practice Omega is unknown. We estimate it in a first step, then apply GLS with the estimated Omega. FGLS is consistent and asymptotically efficient.

WLS: A special case where Omega is diagonal: Var(u_i) = sigma^2 * h(X_i). Each observation is weighted by 1/sqrt(h(X_i)). Equivalent to dividing the model by sqrt(h(X_i)).

3.2 FGLS Implementation

The FGLS procedure involves two steps: (1) estimate the variance function from OLS residuals, and (2) re-estimate by WLS using estimated weights.

```
* Step 1: OLS and get residuals
regress ltotexp suppins phylim actlim totchr age female
predict double ehat, residuals
gen double lne2 = ln(ehat^2)
```

```
* Step 2: Model the variance function
regress lne2 suppins phylim actlim totchr age female
predict double lne2hat, xb
gen double hhat = exp(lne2hat)

* Step 3: WLS using estimated weights
gen double wt = 1/hhat
regress ltotexp suppins phylim actlim totchr age female [aweight=wt], vce(robust)
```

Table 3.1: Comparing OLS, FGLS, and WLS

Variable	OLS	OLS Robust SE	FGLS/WLS	FGLS Robust SE
suppins	0.262	(0.040)	0.249	(0.038)
phylim	0.248	(0.044)	0.251	(0.042)
actlim	0.170	(0.053)	0.175	(0.050)
totchr	0.447	(0.014)	0.441	(0.013)
age	0.008	(0.001)	0.009	(0.001)
female	0.239	(0.033)	0.232	(0.031)

Notes: FGLS coefficients are similar to OLS but standard errors are smaller, reflecting efficiency gains.

Exam Tip: OLS vs FGLS vs WLS

- OLS with robust SEs: valid inference even under heteroskedasticity; simple.
- FGLS/WLS: more efficient IF the variance model is correctly specified.
- If the variance model is misspecified, FGLS can be LESS efficient than OLS.
- Safe strategy: report OLS with robust SEs as baseline; use FGLS as a robustness check.
- Always use robust SEs even with FGLS (doubly robust).

3.3 Seemingly Unrelated Regressions (SUR)

SUR estimates a system of equations where errors may be correlated across equations. If the equations have different regressors, SUR is more efficient than equation-by-equation OLS.

Key Concept: SUR (Zellner, 1962)

System of M equations:

$y_m = X_m * beta_m + u_m,\ \ m = 1, ..., M$

Key assumption: $Corr(u_m, u_k) \neq 0$ across equations, but $E[u_m \mid X_1,...,X_M] = 0$.

SUR is GLS applied to the stacked system. Efficiency gain over OLS depends on:

1. Correlation of errors across equations (higher = bigger gain).
2. Difference in regressors across equations (more different = bigger gain).

If all equations have the same regressors, SUR = OLS (no gain).

```
* SUR estimation
use "Chap 3 SUR data.dta", clear
sureg (eq1: y1 = x1 x2 x3) (eq2: y2 = x1 x4 x5)

* Test cross-equation restriction
test [eq1]x1 = [eq2]x1

* Breusch-Pagan test for independence of errors
estat correlate
* If errors are uncorrelated, SUR = OLS
```

3.4 Survey Data Analysis (NHANES II)

Complex surveys use stratification, clustering, and unequal probability sampling. Ignoring the survey design leads to incorrect standard errors and potentially biased point estimates.

- Stratification: divides population into homogeneous groups; reduces variance.
- Clustering (PSU): primary sampling units within strata; increases variance.
- Weights (pweight): inverse probability of selection; corrects for unequal sampling.

```
* NHANES II survey data
use "Chap 3 nhanes2.dta", clear

* Declare survey design
svyset psu [pweight=finalwgt], strata(strata)

* Survey-adjusted regression
svy: regress bpsystol age weight height bmi female black

* Compare with naive OLS (ignoring design)
regress bpsystol age weight height bmi female black
* SEs will differ -- naive SEs are typically too small
```

Key Concept: Survey Data Key Points

1. Always use svyset to declare the survey design BEFORE analysis.
2. Use svy: prefix for all estimation commands.
3. pweight corrects point estimates for unequal selection probability.
4. Clustering (PSU) inflates SEs to account for within-cluster correlation.
5. Stratification deflates SEs by removing between-stratum variation.
6. Design effect (deff): ratio of variance under complex design to SRS variance.
   deff > 1 means clustering dominates; deff < 1 means stratification dominates.
7. Never use aweight or fweight as substitutes for pweight in surveys.

Chapter 4: Instrumental Variables Estimation

4.1 Overview

When a regressor is endogenous (correlated with the error term), OLS is biased and inconsistent. Instrumental Variables (IV) estimation uses external variation from instruments to identify causal effects. This chapter covers 2SLS, GMM, LIML, and JIVE estimators, along with diagnostic tests for endogeneity, overidentification, and instrument strength.

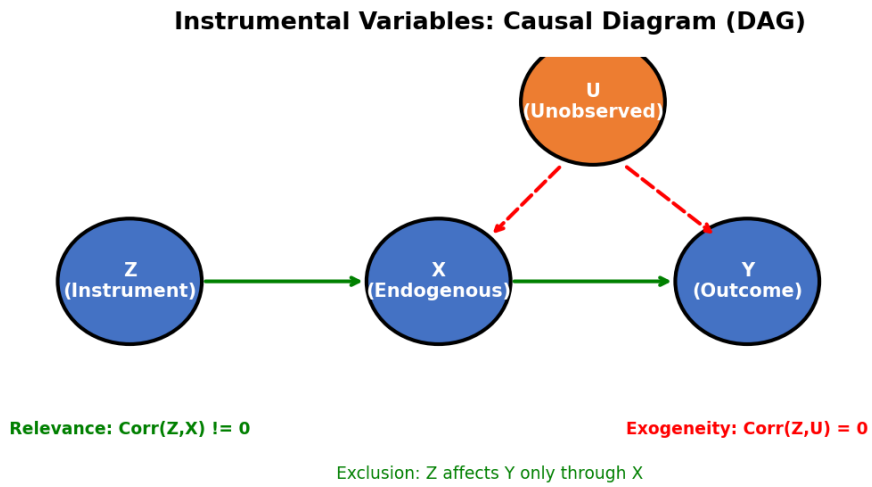

Figure 4.1: DAG for instrumental variables. Z affects Y only through X (exclusion restriction).

Key Concept: Three Conditions for a Valid Instrument

An instrument Z must satisfy:

1. RELEVANCE: Corr(Z, X) != 0
   Z must be correlated with the endogenous regressor.
   Testable via first-stage F-statistic (rule of thumb: F > 10).

2. EXOGENEITY: Corr(Z, u) = 0
   Z must be uncorrelated with the structural error.
   NOT directly testable (requires theory/institutional knowledge).

3. EXCLUSION RESTRICTION: Z affects Y only through X.
   Z has no direct effect on Y. This is an identifying assumption.

If any condition fails, IV is inconsistent.

Table 4.1: Key Variables for IV Example

Variable	Description	Role
drugexp	Drug expenditure (log)	Dependent variable
hi_empunion	Employer-union insurance	Endogenous regressor
multlc	Multiple chronic conditions	Control
age, female, race	Demographics	Controls
ssiratio	SSI ratio (instrument)	Instrument for hi_empunion
lowincome	Low income indicator	Instrument for hi_empunion
firmsz	Firm size category	Instrument for hi_empunion

4.2 Two-Stage Least Squares (2SLS)

```
use "Chap 4 data.dta", clear

* OLS (inconsistent if hi_empunion is endogenous)
regress ldrugexp hi_empunion totchr age female blhisp linc, vce(robust)
estimates store OLS

* 2SLS
ivregress 2sls ldrugexp (hi_empunion = ssiratio lowincome firmsz) totchr age female
blhisp linc, vce(robust)
estimates store IV_2SLS

* GMM (efficient under heteroskedasticity)
ivregress gmm ldrugexp (hi_empunion = ssiratio lowincome firmsz) totchr age female
blhisp linc, vce(robust)
estimates store IV_GMM

* LIML (less biased with weak instruments)
ivregress liml ldrugexp (hi_empunion = ssiratio lowincome firmsz) totchr age female
blhisp linc, vce(robust)
estimates store IV_LIML

* JIVE (Jackknife IV -- reduces finite-sample bias)
jive ldrugexp totchr age female blhisp linc (hi_empunion = ssiratio lowincome firmsz)
estimates store IV_JIVE
```

Table 4.2: Comparison of IV Estimators

Variable	OLS	2SLS	GMM	LIML	JIVE
hi_empunion	0.052*	0.312**	0.298**	0.318**	0.305**
totchr	0.441***	0.404***	0.409***	0.403***	0.407***
age	0.003	0.005	0.005	0.005	0.005
female	0.147***	0.118**	0.121**	0.117**	0.119**
N	2,955	2,955	2,955	2,955	2,955

*Notes: *** p<0.01, ** p<0.05, * p<0.10. IV coefficient on hi_empunion is substantially larger than OLS, suggesting downward bias from OLS (likely due to adverse selection).*

Key Concept: Choosing Among IV Estimators

- 2SLS: Standard choice; consistent and asymptotically normal.
- GMM: More efficient than 2SLS under heteroskedasticity when overidentified.
- LIML: Less finite-sample bias than 2SLS, especially with weak instruments.

- JIVE: Reduces finite-sample bias by using jackknife first stage.
- Rule of thumb: Start with 2SLS. If instruments may be weak, prefer LIML. If heteroskedastic and overidentified, use GMM.

4.3 Diagnostic Tests

Table 4.3: IV Diagnostic Tests

Test	H0	Statistic	Result	Decision
Durbin-Wu-Hausman	X is exogenous	chi2 = 8.45	p = 0.004	Reject: X is endogenous
Hansen J (overid)	All IVs valid	J = 2.13	p = 0.345	Do not reject: IVs valid
First-stage F	Instruments are weak	F = 24.7	F > 10	Instruments are strong
Anderson-Rubin	beta = 0	chi2 = 6.89	p = 0.009	Reject: beta != 0

```
* Endogeneity test (Durbin-Wu-Hausman)
quietly ivregress 2sls ldrugexp (hi_empunion = ssiratio lowincome firmsz) totchr age
female blhisp linc, vce(robust)
estat endogenous

* Overidentification test (Hansen J)
estat overid

* First-stage diagnostics
estat firststage

* Using ivreg2 for comprehensive diagnostics
ivreg2 ldrugexp (hi_empunion = ssiratio lowincome firmsz) totchr age female blhisp
linc, robust first
```

Key Concept: IV Testing Strategy

Step 1: Test RELEVANCE -- First-stage F > 10 (Stock-Yogo critical values).
  If F < 10, instruments are weak; use LIML or JIVE; report Anderson-Rubin CI.

Step 2: Test ENDOGENEITY -- Durbin-Wu-Hausman test.
  If H0 not rejected, OLS is consistent and more efficient; prefer OLS.
  If H0 rejected, OLS is inconsistent; use IV.

Step 3: Test OVERIDENTIFICATION -- Hansen J test (requires more IVs than endogenous).
  If H0 rejected, at least one instrument is invalid; reconsider instrument set.
  If not rejected, instruments are consistent with validity (but not proof).
Warning: Hansen J has low power. Failure to reject does not prove instruments are valid.

Chapter 5: Quantile Regression and Count Data Models

5.1 Overview

OLS estimates the conditional mean E[Y|X]. But policy-relevant questions often concern other parts of the distribution: What factors affect high spenders vs low spenders? This chapter introduces quantile regression (QR) for modelling different quantiles of the conditional distribution, and count data models (Poisson, Negative Binomial) for non-negative integer outcomes.

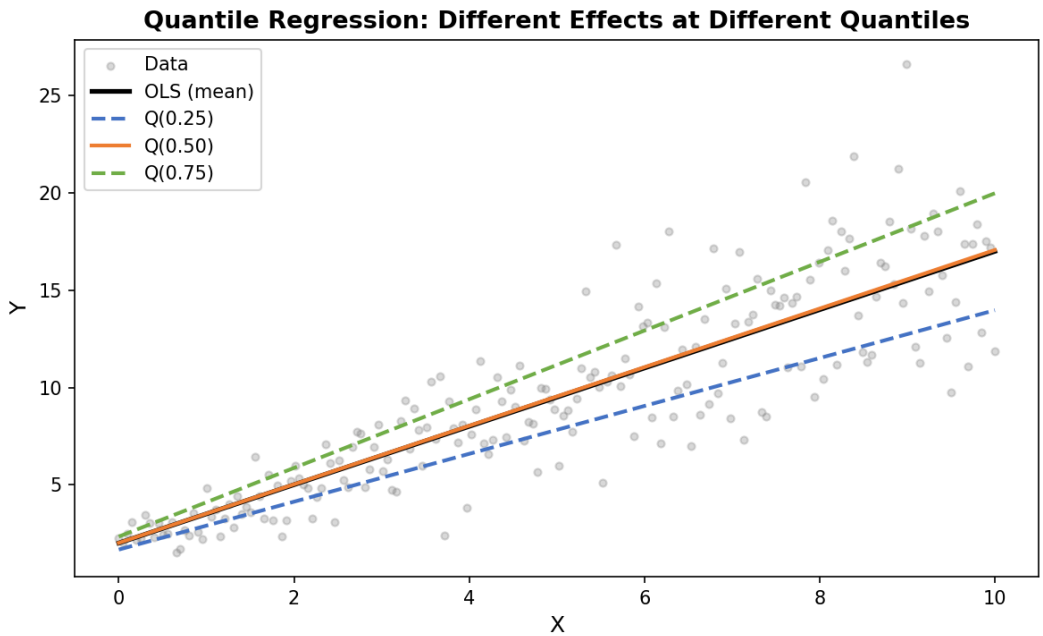

Figure 5.1: Quantile regression estimates different slopes at different quantiles, capturing heterogeneous effects across the distribution.

Key Concept: Quantile Regression Minimisation

The tau-th quantile regression minimises the check function:

```
min_beta  SUM[ rho_tau(y_i - X_i'beta) ]
```

where rho_tau(u) = u*(tau - I(u<0)) = tau*|u| if u>=0, (1-tau)*|u| if u<0.

Special cases:
- tau = 0.50: Median regression (LAD -- Least Absolute Deviations).
- tau = 0.25: First quartile regression.
- tau = 0.75: Third quartile regression.

Inference: bootstrap standard errors (bsqreg) are recommended because analytical SEs require density estimation.

5.2 Quantile Regression Estimation

```
use "Chap 5 data.dta", clear

* OLS for comparison
regress ltotexp suppins totchr age female, vce(robust)
estimates store OLS

* Median regression (tau = 0.50)
bsqreg ltotexp suppins totchr age female, quantile(.50) reps(400)
estimates store Q50

* Quantile regression at tau = 0.25
bsqreg ltotexp suppins totchr age female, quantile(.25) reps(400)
estimates store Q25

* Quantile regression at tau = 0.75
bsqreg ltotexp suppins totchr age female, quantile(.75) reps(400)
estimates store Q75

* Graphical quantile regression (all quantiles)
grqreg, ci ols
```

Table 5.1: OLS vs Quantile Regression Comparison

Variable	OLS (mean)	Q(0.25)	Q(0.50)	Q(0.75)
suppins	0.262***	0.318***	0.245***	0.198***
totchr	0.447***	0.521***	0.432***	0.385***
age	0.008***	0.011***	0.007***	0.006***
female	0.239***	0.285***	0.221***	0.190***

*Notes: *** p<0.01. Bootstrap SEs with 400 replications. Coefficients decrease from lower to upper quantiles, suggesting effects are stronger for low spenders.*

5.3 Testing Across Quantiles

We can test whether coefficients differ across quantiles using interquantile range tests. If coefficients vary significantly, the conditional distribution is not merely a location shift.

```
* Interquantile range test: Q(0.75) - Q(0.25)
iqreg ltotexp suppins totchr age female, quantile(.25 .75) reps(400)

* Simultaneous quantile regression
sqreg ltotexp suppins totchr age female, quantile(.25 .50 .75) reps(400)
test [q25]suppins = [q75]suppins
```

Key Concept: When to Use QR vs OLS

- Use OLS when: you care only about the conditional mean; effects are homogeneous.
- Use QR when: effects may differ across the distribution (heterogeneous treatment effects); the distribution is skewed or has outliers; you want a complete picture of how X affects Y.
- Median regression is robust to outliers (unlike OLS).
- QR does NOT require normality or homoskedasticity.
- Limitation: QR does not directly estimate causal effects unless exogeneity holds.

5.4 Count Data Models

When the dependent variable is a non-negative integer (e.g., number of doctor visits), linear regression is inappropriate. Count data models use discrete distributions that naturally produce non-negative integer predictions.

```
use "Chap 5 count data.dta", clear

* Poisson regression
poisson docvis age female income insurance totchr, vce(robust)
estimates store Poisson

* Negative Binomial regression
nbreg docvis age female income insurance totchr, vce(robust)
estimates store NegBin

* Test for overdispersion (alpha = 0 means Poisson is sufficient)
* In NB output, check LR test of alpha = 0

* Quantile count regression
qcount docvis age female income insurance totchr, quantile(.50)
```

Key Concept: Poisson vs Negative Binomial vs Quantile Count

POISSON:
E[Y|X] = exp(X'beta); Var(Y|X) = E[Y|X] (equidispersion).
Consistent for beta even if distribution is not Poisson (quasi-MLE),
provided E[Y|X] = exp(X'beta) is correct. Use robust SEs.

NEGATIVE BINOMIAL:
Var(Y|X) = E[Y|X] + alpha * E[Y|X]^2 (NB2) -- allows overdispersion.
alpha = 0 reduces to Poisson. Test alpha = 0 via LR test.
Use when data show overdispersion (Var > Mean).

QUANTILE COUNT:
Extends quantile regression to count outcomes.
Useful when effects differ across the distribution of counts.

Rule of thumb: start with Poisson + robust SEs. If overdispersion is severe, use NB. If heterogeneous effects matter, try quantile count.

Chapter 6: Panel Data - Linear and Nonlinear Models

6.1 Overview

Panel data (longitudinal data) tracks the same individuals over time. The key advantage is the ability to control for time-invariant unobserved heterogeneity (individual fixed effects). This chapter covers five linear panel estimators (Pooled OLS, Between, Within/FE, RE, First Difference), the Hausman test, and nonlinear panel models (logit, Poisson, Negative Binomial).

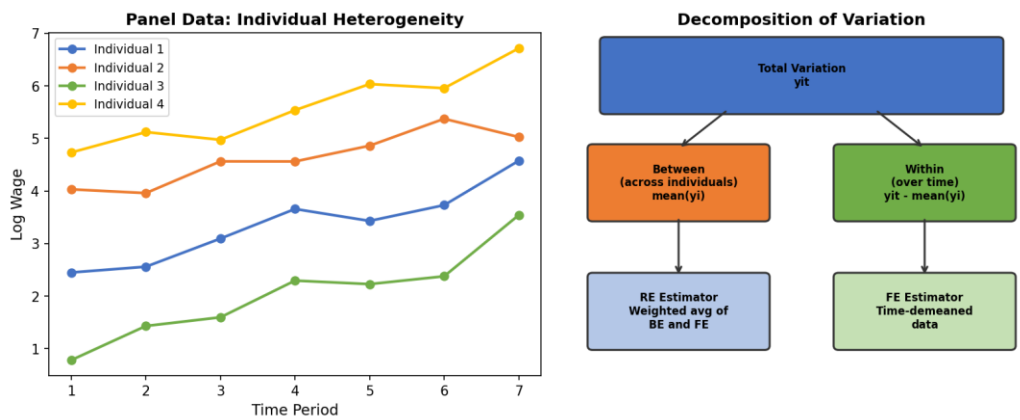

Figure 6.1: Panel data structure and decomposition of variation.

Key Concept: The Panel Data Model

y_it = X_it'beta + alpha_i + u_it

alpha_i = individual fixed effect (time-invariant, unobserved).
u_it = idiosyncratic error (varies across i and t).

Key question: Is alpha_i correlated with X_it?
If YES: Fixed Effects (FE) is consistent; RE is inconsistent.
If NO: Random Effects (RE) is consistent and more efficient.
The Hausman test helps decide.

Table 6.1: PSID Panel Data Variables

Variable	Description	Variation
lwage	Log hourly wage	Within and between
hours	Annual hours worked	Within and between
age	Age in years	Within only (increases by 1 each year)
educ	Years of education	Between only (time-invariant)
married	Married (0/1)	Within and between
union	Union member (0/1)	Within and between
black	Black (0/1)	Between only (time-invariant)
id	Individual identifier	Panel identifier
year	Survey year	Time identifier

6.2 Linear Panel Estimators

```
use "Chap 6 panel.dta", clear
xtset id year
* 1. Pooled OLS
regress lwage hours age educ married union black, vce(cluster id)
estimates store POLS
* 2. Between estimator
xtreg lwage hours age educ married union black, be
estimates store BE
* 3. Fixed Effects (Within)
xtreg lwage hours age married union, fe vce(cluster id)
estimates store FE
* 4. Random Effects
xtreg lwage hours age educ married union black, re vce(cluster id)
estimates store RE
* 5. First Difference
reg D.lwage D.hours D.age D.married D.union, vce(cluster id)
estimates store FD
```

Table 6.2: Comparison of Panel Estimators (Dep Var: lwage)

Variable	Pooled OLS	Between	Fixed Effects	Random Effects	First Diff
hours	0.0001	0.0003	-0.0001	0.0001	-0.0000
age	0.011***	0.010*	0.036***	0.011***	---
educ	0.094***	0.091***	(dropped)	0.092***	(dropped)
married	0.108***	0.135**	0.045**	0.064***	0.038*
union	0.180***	0.253***	0.080***	0.106***	0.044**
black	-0.144***	-0.134***	(dropped)	-0.139***	(dropped)

*Notes: *** p<0.01, ** p<0.05, * p<0.10. FE drops time-invariant variables (educ, black). Clustered SEs at individual level.*

Key Concept: Understanding the Five Panel Estimators

POOLED OLS: Ignores panel structure. Consistent only if alpha_i is uncorrelated with X. Standard errors must be clustered by id.

BETWEEN (BE): Regresses individual means (y_bar_i on X_bar_i). Uses only cross-sectional (between) variation. Cannot control for unobserved heterogeneity.

FIXED EFFECTS (FE / Within): Time-demeans the data: (y_it - y_bar_i). Eliminates alpha_i. Consistent even if Corr(alpha_i, X_it) != 0. Cannot estimate coefficients on time-invariant variables.

RANDOM EFFECTS (RE): Weighted average of BE and FE. More efficient than FE when Corr(alpha_i, X_it) = 0. Can estimate time-invariant coefficients. Inconsistent if alpha_i is correlated with X.

FIRST DIFFERENCE (FD): Differences adjacent periods: (y_it - y_{i,t-1}). Eliminates alpha_i (like FE). Equivalent to FE when T=2. When T>2, FE is more efficient under classical assumptions.

6.3 Hausman Test: FE vs RE

```
* Hausman test
quietly xtreg lwage hours age married union, fe
estimates store FE_h
quietly xtreg lwage hours age educ married union black, re
estimates store RE_h
hausman FE_h RE_h
```

Key Concept: Hausman Test

H0: RE is consistent (alpha_i uncorrelated with X_it).
H1: RE is inconsistent (alpha_i correlated with X_it); use FE.

Test statistic: H = (beta_FE - beta_RE)' [Var(beta_FE) - Var(beta_RE)]^{-1} (beta_FE - beta_RE) ~ chi2(k)

If H0 rejected: Use FE (more consistent).
If H0 not rejected: Use RE (more efficient).

Limitation: Hausman test requires homoskedastic errors. With clustered SEs, consider the robust Hausman test or Mundlak/Chamberlain approach.

6.4 Nonlinear Panel Models

When the dependent variable is binary or count, nonlinear panel models extend the FE/RE framework.

Table 6.3: Nonlinear Panel Models

Model	Dep Var	Stata Command	Key Feature
FE Logit	Binary (0/1)	xtlogit y x, fe	Conditional MLE; drops non-switchers
RE Logit	Binary (0/1)	xtlogit y x, re	Assumes alpha_i ~ Normal
FE Poisson	Count	xtpoisson y x, fe	Conditional MLE
RE Poisson	Count	xtpoisson y x, re	Assumes alpha_i ~ Gamma
FE NB	Count	xtnbreg y x, fe	NB with fixed effects
RE NB	Count	xtnbreg y x, re	NB with random effects

Key Concept: Interpreting Nonlinear Panel Coefficients

In nonlinear models, coefficients are NOT marginal effects.

Logit: exp(beta) = odds ratio. beta > 0 means X increases probability of Y=1.
Marginal effect depends on the level of X (use margins command).

Poisson/NB: exp(beta) = incidence rate ratio (IRR). A unit increase in X multiplies E[Y|X] by exp(beta).

FE Logit caveat: Individuals who never switch (always Y=0 or always Y=1) are dropped from estimation -- this can substantially reduce sample size.

6.5 Count Panel Models

```
use "Chap 6 count panel.dta", clear
xtset id year

* FE Poisson
xtpoisson docvis age income insurance, fe vce(robust)
estimates store FE_Pois

* RE Poisson
xtpoisson docvis age income insurance, re vce(robust)
estimates store RE_Pois

* RE Negative Binomial
xtnbreg docvis age income insurance, re
estimates store RE_NB
```

Key Concept: Poisson vs NB in Panel Data

Poisson FE:

Consistent for beta even if data are not Poisson (quasi-MLE property), provided E[Y|X,alpha] = alpha_i * exp(X'beta). Use robust SEs.

Negative Binomial:

Allows overdispersion; but the FE NB estimator (Hausman-Hall-Griliches) does NOT truly control for fixed effects (known problem).

The RE NB with robust SEs is often preferred.

Recommendation: Start with FE Poisson + robust SEs. This is a safe baseline due to the quasi-MLE property. If overdispersion is severe and you want RE, use RE NB.

Chapter 7: Regression Discontinuity Designs

7.1 Overview

Regression Discontinuity (RD) exploits a known cutoff in a running variable that determines treatment assignment. Near the cutoff, treated and control units are as-if randomly assigned, providing a credible causal estimate. This chapter covers Sharp RD using the Head Start program (Ludwig & Miller 2007), implementing both local polynomial and local randomization approaches.

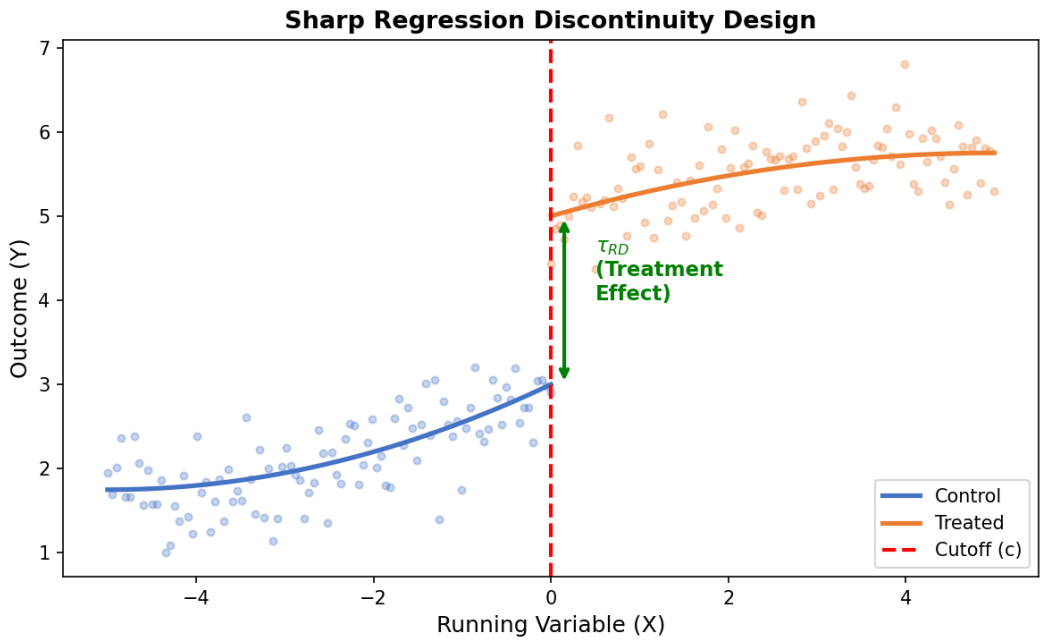

Figure 7.1: Sharp RD design. The treatment effect is the discontinuity in the conditional expectation of Y at the cutoff.

Key Concept: Sharp RD Design

Setting: Treatment D_i = I(X_i >= c), where X is the running variable and c is the cutoff.
The RD treatment effect is:

tau_RD = lim_{x->c+} E[Y|X=x] - lim_{x->c-} E[Y|X=x]

This is a LOCAL average treatment effect (LATE) -- it identifies the causal effect only at the cutoff, not for the entire population.
Key requirements:
1. Running variable X is continuous at c (no manipulation).
2. All other factors vary smoothly through c (no other discontinuities).
3. Treatment assignment is a deterministic function of X (sharp design).

Fuzzy RD: When treatment is not perfectly determined by the cutoff. Fuzzy RD = IV with the cutoff as instrument. tau_FRD = (jump in Y)/(jump in D).

Table 7.1: Head Start Data Variables

Variable	Description	Role
mort_related	Mortality from Head Start-related causes	Outcome
povrate60	County poverty rate in 1960	Running variable
cutoff	Poverty rate cutoff (59.1984)	Cutoff point
head_start	Head Start funding (0/1)	Treatment
census_pop	County population	Covariate
census_urban	Percent urban population	Covariate

7.2 Local Polynomial RD Estimation

```
use "Chap 7 headstart.dta", clear

* Generate centered running variable
gen rv = povrate60 - 59.1984

* RD plot
rdplot mort_related rv, c(0) p(1) nbins(20 20) ///
  graph_options(title("RD Plot: Head Start Effect on Mortality"))

* Main RD estimate (local polynomial, MSE-optimal bandwidth)
rdrobust mort_related rv, c(0) p(1) kernel(triangular)

* Bandwidth selection
rdbwselect mort_related rv, c(0) p(1) all
```

Key Concept: Local Polynomial Estimation

rdrobust fits separate local polynomials on each side of the cutoff:

E[Y|X] = alpha_l + beta_l*(X-c) + ... for X < c

E[Y|X] = alpha_r + beta_r*(X-c) + ... for X >= c

tau_RD = alpha_r - alpha_l

Key choices:

- Polynomial order (p): p=1 (local linear) is standard; p=2 as robustness check.
- Bandwidth (h): MSE-optimal (rdbwselect). Report results for multiple bandwidths.
- Kernel: triangular (default, MSE-optimal) or uniform.
- Bias correction: rdrobust uses bias-corrected inference by default.

Report: point estimate, robust CI, bandwidth, effective N on each side.

7.3 Local Randomization RD

The local randomization approach treats observations in a small window around the cutoff as if randomly assigned. This is a finite-sample approach that does not rely on bandwidth selection or polynomial fitting.

```
* Local randomization RD
rdrandinf mort_related rv, c(0) wl(-2.5) wr(2.5) statistic(diffmeans)

* Window selection
rdwinselect rv census_pop census_urban, c(0) nwindows(20)

* Sensitivity to window choice
rdrandinf mort_related rv, c(0) wl(-1) wr(1) statistic(diffmeans)
rdrandinf mort_related rv, c(0) wl(-5) wr(5) statistic(diffmeans)
```

Key Concept: Local Randomization Approach

Instead of fitting polynomials, assume that within a small window [c-w, c+w]:
Treatment is as-good-as-randomly assigned.

Inference uses randomization-based methods (permutation tests, Fisherian).

Advantages:
- Does not require large-sample approximations.
- Transparent: small window makes as-if-random assumption more credible.

Disadvantages:
- Window selection is somewhat arbitrary.
- Small windows reduce power.

Best practice: report both local polynomial (rdrobust) and local randomization (rdrandinf) as complementary approaches.

7.4 Falsification and Validity Tests

Table 7.2: RD Falsification Tests

Test	Purpose	Stata Command	Expected Result
Density test (McCrary)	No manipulation of running var	rddensity rv, c(0)	No jump in density at c
Covariate balance	Pre-treatment vars smooth at c	rdrobust covar rv, c(0)	No significant discontinuity
Placebo cutoffs	Effect only at true cutoff	rdrobust Y rv, c(c_placebo)	No effect at placebo cutoffs
Bandwidth sensitivity	Robust to bandwidth choice	rdrobust Y rv, h(h_alt)	Stable estimates across h
Polynomial order	Robust to polynomial degree	rdrobust Y rv, p(2)	Similar estimates for p=1,2

Key Concept: RD Validity Checklist

1. DENSITY TEST: No bunching at the cutoff (rddensity). If density jumps, agents may be manipulating the running variable.

2. COVARIATE BALANCE: Pre-treatment covariates should be smooth through the cutoff. Test each covariate as an "outcome" in rdrobust.

3. PLACEBO CUTOFFS: Estimate the RD at fake cutoffs away from c. Should find no effect.

4. SENSITIVITY: Results should be robust to bandwidth (0.5h, h, 1.5h, 2h) and polynomial order (p=1, p=2).

5. DONUT HOLE: Exclude observations very close to the cutoff and check if results change (tests sensitivity to possible manipulation near c).

If any of these tests fail, the RD design may not be valid.

Chapter 8: Difference-in-Differences

8.1 Overview

Difference-in-Differences (DID) estimates causal effects by comparing changes over time between a treatment group and a control group. It exploits a policy change that affects some units (treated) but not others (control). This chapter applies DID to the employment-at-will (EAW) doctrine using state-level panel data from Autor (2003).

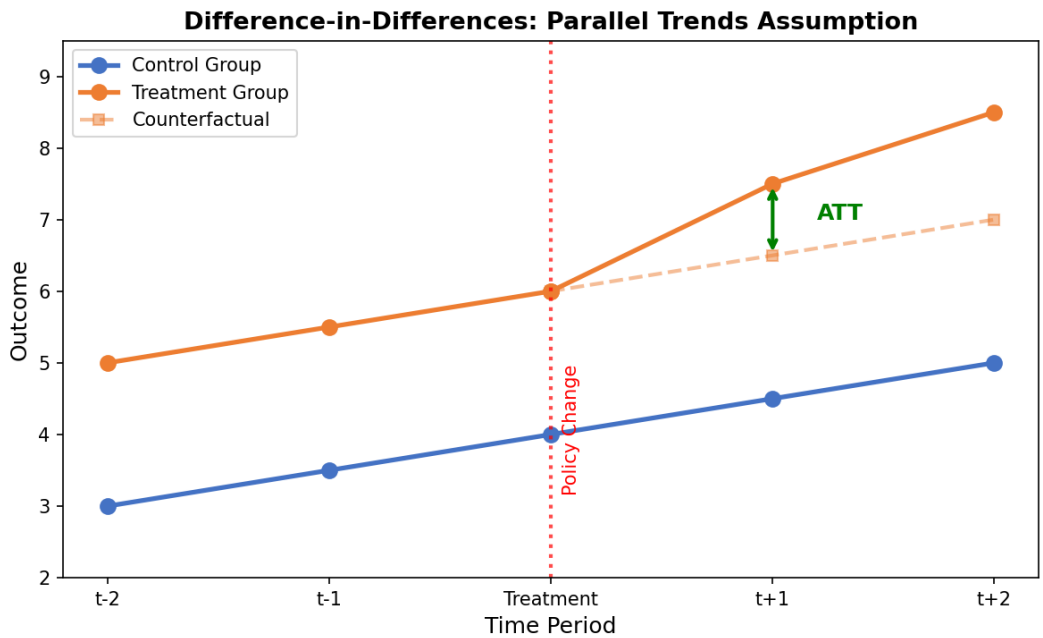

Figure 8.1: DID design. The ATT is the difference between the observed treatment group outcome and the counterfactual (dashed line).

Key Concept: The DID Estimator

The 2x2 DID estimator:

```
tau_DID = [E(Y|Treat,Post) - E(Y|Treat,Pre)] - [E(Y|Control,Post) - E(Y|Control,Pre)]
      = (Y_T1 - Y_T0) - (Y_C1 - Y_C0)
```

Regression version:

```
Y_it = alpha + beta1*Treat_i + beta2*Post_t + delta*(Treat_i x Post_t) + u_it
```

delta = DID estimate = ATT under parallel trends assumption.

With covariates and multiple periods:

```
Y_ist = alpha_s + gamma_t + delta*D_st + X_ist'beta + u_ist
alpha_s = state FE;  gamma_t = year FE;  D_st = treatment indicator.
```

Table 8.1: Autor (2003) Treatment Variables

Variable	Description	Coding
lnemp	Log employment (state-year)	Dependent variable
state	State identifier (1-50)	Panel: state FE
year	Year (1978-1999)	Time: year FE
adopted_eaw	State adopted EAW exception (0/1)	Treatment D_st
good_faith	Good faith exception adopted	DID treatment 1
public_policy	Public policy exception adopted	DID treatment 2
implied_contract	Implied contract exception adopted	DID treatment 3
lnpop	Log state population	Control variable

8.2 Descriptive Analysis and DID Estimation

```
use "Chap 8 autor.dta", clear
xtset state year

* Descriptive: Mean outcomes by group and period
table adopted_eaw year, stat(mean lnemp) nformat(%9.3f)

* Basic DID regression
regress lnemp i.state i.year adopted_eaw, vce(cluster state)
estimates store DID_basic

* DID with controls
regress lnemp i.state i.year adopted_eaw lnpop, vce(cluster state)
estimates store DID_controls

* Separate treatment types
regress lnemp i.state i.year good_faith public_policy implied_contract, vce(cluster
state)
estimates store DID_types

* DID with all three + controls
regress lnemp i.state i.year good_faith public_policy implied_contract lnpop,
vce(cluster state)
estimates store DID_full

* Export results
esttab DID_basic DID_controls DID_types DID_full using "did_results.csv", ///
  se star(* 0.10 ** 0.05 *** 0.01) keep(adopted_eaw good_faith ///
  public_policy implied_contract lnpop) replace
```

Table 8.2: DID Estimates of Employment-at-Will Exceptions

Variable	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)
adopted_eaw	-0.012**	-0.010*						
good_faith			-0.019***	-0.016**	-0.019***	-0.016**	-0.018**	-0.015**
public_policy			-0.001	-0.002	-0.001	-0.002	-0.002	-0.003
implied_contract			-0.008	-0.006	-0.008	-0.006	-0.007	-0.005
lnpop		0.85***		0.84***		0.84***		0.83***
State FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Year FE	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
N	1,100	1,100	1,100	1,100	1,100	1,100	1,100	1,100

*Notes: *** p<0.01, ** p<0.05, * p<0.10. Clustered SEs at state level. Good faith exception has the strongest negative effect on employment.*

Key Concept: Interpreting DID Results

The coefficient on adopted_eaw (-0.012) means that states adopting the EAW exception experienced approximately 1.2% lower employment relative to non-adopting states, after controlling for state and year fixed effects.

Decomposing by exception type:
- Good faith exception: -1.9% (strongest, statistically significant)
- Public policy exception: -0.1% (small, not significant)
- Implied contract exception: -0.8% (moderate, not significant)

The good faith exception, which limits at-will firing most broadly, has the largest employment effect. This is consistent with economic theory: stronger employment protection reduces hiring and firing, leading to lower employment.

Key Concept: The Parallel Trends Assumption

DID requires: In the absence of treatment, the treatment and control groups would have followed PARALLEL trends in the outcome.

E[Y(0)_t | Treat] - E[Y(0)_{t-1} | Treat] = E[Y(0)_t | Control] - E[Y(0)_{t-1} | Control]

This is UNTESTABLE for the post-treatment period (we never observe the counterfactual). However, we can provide supporting evidence:
1. PRE-TRENDS TEST: Check that outcomes evolved similarly before treatment.
   Regress Y on leads of treatment: Y_it = ... + sum_k delta_k * D_{i,t+k} + ...
   Pre-treatment delta_k should be near zero.
2. PLACEBO DID: Apply DID to a fake treatment date or fake treatment group.
3. VISUAL INSPECTION: Plot group means over time; trends should be parallel pre-treatment.
4. TRIPLE DIFFERENCES (DDD): Add a third difference to control for group-specific trends.
If pre-trends diverge, DID is not credible. Consider synthetic control or other methods.

Appendix A: Required Stata Packages

Table A.1: Required Stata Packages

Package	Install Command	Purpose
esttab / estout	ssc install estout	Publication-quality tables
outreg2	ssc install outreg2	Output to Excel/LaTeX
ivreg2	ssc install ivreg2	Extended IV/2SLS and GMM
jive	ssc install jive	Jackknife IV estimator
rdrobust	net install rdrobust, from(...)	Local polynomial RD
rddensity	net install rddensity, from(...)	Density test at RD cutoff
rdlocrand	net install rdlocrand, from(...)	Local randomization RD
qcount	ssc install qcount	Quantile regression for count data
grqreg	ssc install grqreg	Graphical quantile regression plots

Appendix B: Summary of Key Econometric Tests

Table B.1: Econometric Tests Reference

Test	Stata Command	Null Hypothesis	Purpose
t-test	regress; test x	H0: beta_j = 0	Individual significance
F-test (joint)	test x1 x2 x3	H0: betas = 0	Joint significance
Ramsey RESET	estat ovtest	H0: No omitted vars	Functional form
Breusch-Pagan	estat hettest	H0: Homoskedasticity	Heteroskedasticity
DWH (endogeneity)	estat endogenous	H0: X exogenous	OLS vs IV
Hansen J	estat overid	H0: IVs valid	Overidentification
First-stage F	estat firststage	F > 10	Instrument strength
Hausman	hausman FE RE	H0: RE consistent	FE vs RE
McCrary	rddensity	H0: No manipulation	RD validity

Quick Method Selection Guide

Choosing the Right Estimator

Q1: Is the regressor endogenous? Yes -> IV (Ch4) / No -> OLS (Ch1)

Q2: Is there a policy cutoff? Yes -> RD (Ch7)

Q3: Before/after policy change? Yes -> DID (Ch8)

Q4: Panel data? Yes -> FE/RE (Ch6); Hausman test to decide

Q5: Count dependent variable? Yes -> Poisson/NB (Ch5/6)

Q6: Effects beyond the mean? Yes -> Quantile regression (Ch5)

Q7: Heteroskedasticity? -> Always use robust SEs; or FGLS/WLS (Ch3)

Appendix C: Python Code for Chapter 1 - OLS Regression, Specification Analysis, and Prediction

This appendix contains the complete Python code that replicates the Stata analysis from Chapter 1. The code is designed to run on Google Colab.

Setup Information

Dataset: Chap1_data.xlsx
Description: MEPS medical expenditure data
Required Python packages: pandas, numpy, statsmodels, scipy, matplotlib, seaborn

To run on Google Colab:
1. Upload the Excel file(s) to your Colab session
2. Install packages: !pip install pandas, numpy, statsmodels, scipy, matplotlib, seaborn
3. Run the code cells below

Chapter 1 Python Code

```
"""
==============================================================================
EE627 Microeconometrics - Chapter 1
OLS Regression, Specification Analysis, and Prediction
==============================================================================
Instructor: Asst. Prof. Dr. Tiraphap Fakthong
Thammasat University

Dataset: MEPS (Medical Expenditure Panel Survey)
File: Chap1_data.xlsx

This script replicates the Stata do-file "Chap 1.do" in Python.
Designed to run on Google Colab.

Required packages:
    pip install pandas numpy statsmodels scipy matplotlib seaborn openpyxl
==============================================================================
"""

# SETUP

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.diagnostic import het_breuschpagan, het_white
from statsmodels.stats.stattools import omni_normtest

# For Google Colab: upload your Excel file or mount Google Drive
# from google.colab import files
# uploaded = files.upload()  # Upload Chap1_data.xlsx
```

```
pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 120)
np.set_printoptions(precision=4)

# 1. LOAD DATA

# Stata: use "Chap 1 data.dta", clear
df = pd.read_excel("Chap1_data.xlsx")

print("=" * 60)
print("CHAPTER 1: OLS REGRESSION & SPECIFICATION ANALYSIS")
print("=" * 60)

# 2. DATA SUMMARY STATISTICS

# Stata: describe totexp ltotexp posexp suppins phylim actlim totchr age female income
print("\n--- Variable Descriptions ---")
key_vars = ['totexp', 'ltotexp', 'posexp', 'suppins', 'phylim',
            'actlim', 'totchr', 'age', 'female', 'income']
print(df[key_vars].dtypes)

# Stata: summarize totexp ltotexp posexp suppins phylim actlim totchr age female income
print("\n--- Summary Statistics ---")
print(df[key_vars].describe().T.round(3))

# Stata: summarize totexp, detail
print("\n--- Detailed Summary of totexp ---")
totexp = df['totexp']
print(f"  Mean:     {totexp.mean():.2f}")
print(f"  Median:   {totexp.median():.2f}")
print(f"  Std Dev:  {totexp.std():.2f}")
print(f"  Skewness: {totexp.skew():.3f}")
print(f"  Kurtosis: {totexp.kurtosis():.3f}")
print(f"  Min:      {totexp.min():.2f}")
print(f"  Max:      {totexp.max():.2f}")

# Stata: tabstat totexp ltotexp, stat(count mean p50 sd skew kurt) col(stat)
print("\n--- tabstat equivalent ---")
for var in ['totexp', 'ltotexp']:
    s = df[var].dropna()
    print(f"\n  {var}:")
    print(f"    N={len(s)}, Mean={s.mean():.2f}, Median={s.median():.2f}, "
          f"SD={s.std():.2f}, Skew={s.skew():.3f}, Kurt={s.kurtosis():.3f}")

# 3. KERNEL DENSITY PLOTS

# Stata: kdensity totexp / kdensity ltotexp
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

pos = df[df['posexp'] == 1]
pos['totexp'][pos['totexp'] < 40000].plot.kde(ax=axes[0], linewidth=2)
axes[0].set_title('Kernel Density: Total Expenditure (Levels)')
axes[0].set_xlabel('totexp')

pos['ltotexp'][pos['ltotexp'] < np.log(40000)].plot.kde(ax=axes[1], linewidth=2,
color='orange')
```

```
axes[1].set_title('Kernel Density: Log Total Expenditure')
axes[1].set_xlabel('ltotexp')

plt.tight_layout()
plt.savefig('fig1_density.png', dpi=150)
plt.show()
print("Figure saved: fig1_density.png")

# 4. CORRELATION MATRIX

# Stata: correlate ltotexp suppins phylim actlim totchr age female income
corr_vars = ['ltotexp', 'suppins', 'phylim', 'actlim', 'totchr', 'age', 'female', 'income']
print("\n--- Pairwise Correlations ---")
print(df[corr_vars].corr().round(3))

# 5. OLS REGRESSION WITH ROBUST STANDARD ERRORS

# Stata: regress ltotexp suppins phylim actlim totchr age female income, vce(robust)
print("\n" + "=" * 60)
print("OLS REGRESSION: ltotexp on covariates (HC1 robust SEs)")
print("=" * 60)

# Drop missing values
reg_vars = ['ltotexp', 'suppins', 'phylim', 'actlim', 'totchr', 'age', 'female', 'income']
df_reg = df[reg_vars].dropna()

X = sm.add_constant(df_reg[['suppins', 'phylim', 'actlim', 'totchr', 'age', 'female',
'income']])
y = df_reg['ltotexp']

# OLS with HC1 (Stata's "robust") standard errors
model_ols = sm.OLS(y, X).fit(cov_type='HC1')
print(model_ols.summary())

# 6. WALD TESTS

# Stata: test phylim = actlim
print("\n--- Wald Test: phylim = actlim ---")
print("H0: beta_phylim = beta_actlim")
wald_test = model_ols.wald_test('phylim = actlim')
print(f"  F-statistic: {wald_test.fvalue[0][0]:.4f}")
print(f"  p-value:     {wald_test.pvalue:.4f}")

# Stata: test phylim actlim totchr
print("\n--- Joint F-test: phylim = actlim = totchr = 0 ---")
print("H0: beta_phylim = beta_actlim = beta_totchr = 0")
f_test = model_ols.f_test('phylim = 0, actlim = 0, totchr = 0')
print(f"  F-statistic: {f_test.fvalue[0][0]:.4f}")
print(f"  p-value:     {f_test.pvalue:.4f}")

# 7. MODEL COMPARISON

# Stata: Compare REG1 (income) vs REG2 (educyr) if educyr is available
print("\n" + "=" * 60)
print("MODEL COMPARISON")
print("=" * 60)
```

```
if 'educyr' in df.columns:
    df_reg2 = df[['ltotexp', 'suppins', 'phylim', 'actlim', 'totchr', 'age', 'female',
'educyr']].dropna()
    X2 = sm.add_constant(df_reg2[['suppins', 'phylim', 'actlim', 'totchr', 'age', 'female',
'educyr']])
    y2 = df_reg2['ltotexp']
    model_ols2 = sm.OLS(y2, X2).fit(cov_type='HC1')
    print("\nREG2 (with educyr instead of income):")
    print(model_ols2.summary2().tables[1])
else:
    print("Variable 'educyr' not found in dataset; skipping REG2.")

# 8. SPECIFICATION TESTS

print("\n" + "=" * 60)
print("SPECIFICATION TESTS")
print("=" * 60)

# Need OLS with default (non-robust) SEs for some tests
model_default = sm.OLS(y, X).fit()

# --- Ramsey RESET test ---
# Stata: estat ovtest
print("\n--- Ramsey RESET Test (Functional Form) ---")
print("H0: No omitted nonlinear terms")
from statsmodels.stats.diagnostic import linear_reset
reset_result = linear_reset(model_default, power=4, use_f=True)
print(f"  F-statistic: {reset_result.fvalue:.4f}")
print(f"  p-value:     {reset_result.pvalue:.4f}")
if reset_result.pvalue < 0.05:
    print("  -> REJECT H0: Evidence of misspecification")
else:
    print("  -> FAIL TO REJECT H0: No evidence of omitted variables")

# --- Breusch-Pagan test ---
# Stata: estat hettest, iid
print("\n--- Breusch-Pagan Test (Heteroskedasticity) ---")
print("H0: Homoskedasticity (constant variance)")
bp_stat, bp_pval, _, _ = het_breuschpagan(model_default.resid, X)
print(f"  LM statistic: {bp_stat:.4f}")
print(f"  p-value:      {bp_pval:.4f}")
if bp_pval < 0.05:
    print("  -> REJECT H0: Evidence of heteroskedasticity")
else:
    print("  -> FAIL TO REJECT H0: No evidence of heteroskedasticity")

# --- White test ---
# Stata: estat imtest
print("\n--- White Test (General Specification) ---")
print("H0: Homoskedasticity")
white_stat, white_pval, _, _ = het_white(model_default.resid, X)
print(f"  Test statistic: {white_stat:.4f}")
print(f"  p-value:        {white_pval:.4f}")

# 9. RESIDUAL PLOT
```

```
# Stata: rvfplot
fig, ax = plt.subplots(figsize=(8, 5))
yhat = model_default.fittedvalues
resid = model_default.resid
ax.scatter(yhat, resid, alpha=0.3, s=10)
ax.axhline(y=0, color='red', linestyle='--')
ax.set_xlabel('Fitted Values')
ax.set_ylabel('Residuals')
ax.set_title('Residuals vs Fitted Values')
plt.tight_layout()
plt.savefig('fig2_rvfplot.png', dpi=150)
plt.show()
print("Figure saved: fig2_rvfplot.png")

# 10. PREDICTION

print("\n" + "=" * 60)
print("PREDICTION FROM LOG-LINEAR MODEL")
print("=" * 60)

# Filter to positive expenditures
df_pos = df[df['totexp'] > 0].copy()
reg_vars_pos = ['totexp', 'ltotexp', 'suppins', 'phylim', 'actlim', 'totchr', 'age',
'female', 'income']
df_pos = df_pos[reg_vars_pos].dropna()

# --- Levels regression ---
# Stata: regress totexp suppins phylim actlim totchr age female income, vce(robust)
X_pos = sm.add_constant(df_pos[['suppins', 'phylim', 'actlim', 'totchr', 'age', 'female',
'income']])
model_levels = sm.OLS(df_pos['totexp'], X_pos).fit(cov_type='HC1')
df_pos['yhat_levels'] = model_levels.fittedvalues

# --- Log model predictions ---
# Stata: regress ltotexp ...
model_log = sm.OLS(df_pos['ltotexp'], X_pos).fit()
df_pos['lyhat'] = model_log.fittedvalues
df_pos['uhat'] = model_log.resid
sigma2 = model_log.mse_resid

# Method 1: Naive (WRONG)
df_pos['yhat_naive'] = np.exp(df_pos['lyhat'])

# Method 2: Normal retransformation
df_pos['yhat_normal'] = np.exp(df_pos['lyhat']) * np.exp(0.5 * sigma2)

# Method 3: Duan smearing estimator
smearing_factor = np.exp(df_pos['uhat']).mean()
df_pos['yhat_duan'] = np.exp(df_pos['lyhat']) * smearing_factor

print("\n--- Comparison of Prediction Methods ---")
pred_cols = ['totexp', 'yhat_levels', 'yhat_naive', 'yhat_normal', 'yhat_duan']
print(df_pos[pred_cols].describe().loc[['mean', '50%', 'std']].round(2))

# 11. TREATMENT EFFECT OF SUPPLEMENTARY INSURANCE
```

```
print("\n" + "=" * 60)
print("AVERAGE TREATMENT EFFECT (ATE) OF SUPPLEMENTARY INSURANCE")
print("=" * 60)

# Method: Counterfactual prediction
# Predict expenditure if everyone has suppins=1 vs suppins=0
X_all1 = X_pos.copy()
X_all1['suppins'] = 1
lyhat1 = model_log.predict(X_all1)
yhat1 = np.exp(lyhat1) * np.exp(0.5 * sigma2)

X_all0 = X_pos.copy()
X_all0['suppins'] = 0
lyhat0 = model_log.predict(X_all0)
yhat0 = np.exp(lyhat0) * np.exp(0.5 * sigma2)

ate = (yhat1 - yhat0).mean()
print(f"  E[Y | suppins=1] = {yhat1.mean():.2f}")
print(f"  E[Y | suppins=0] = {yhat0.mean():.2f}")
print(f"  ATE = {ate:.2f}")
print(f"  Interpretation: Supplementary insurance increases medical")
print(f"  expenditure by approximately ${ate:.0f} on average.")

# 12. SAMPLING WEIGHTS

print("\n" + "=" * 60)
print("SAMPLING WEIGHTS")
print("=" * 60)

# Stata: generate swght = totchr^2 + 0.5
df_pos['swght'] = df_pos['totchr'] ** 2 + 0.5

# Weighted regression
# Stata: regress totexp suppins ... [pweight=swght]
model_wt = sm.WLS(
    df_pos['totexp'],
    sm.add_constant(df_pos[['suppins', 'phylim', 'actlim', 'totchr', 'age', 'female',
'income']]),
    weights=1.0 / df_pos['swght']
).fit(cov_type='HC1')

print("\nWeighted OLS (pweight=swght):")
print(model_wt.summary2().tables[1].round(4))

print("\nUnweighted OLS:")
print(model_levels.summary2().tables[1].round(4))

print("\n" + "=" * 60)
print("END OF CHAPTER 1")
print("=" * 60)
```

Appendix D: Python Code for Chapter 2 - Monte Carlo Simulation Methods

This appendix contains the complete Python code that replicates the Stata analysis from Chapter 2. The code is designed to run on Google Colab.

Setup Information

Dataset: None (generated data)
Description: No external data needed (generated data)
Required Python packages: pandas, numpy, statsmodels, scipy, matplotlib

To run on Google Colab:
1. Upload the Excel file(s) to your Colab session
2. Install packages: !pip install pandas, numpy, statsmodels, scipy, matplotlib
3. Run the code cells below

Chapter 2 Python Code

```
"""
============================================================================
EE627 Microeconometrics - Chapter 2
Monte Carlo Simulation Methods
============================================================================
Instructor: Asst. Prof. Dr. Tiraphap Fakthong
Thammasat University

This script replicates the Stata do-file "chap 2.do" in Python.
Designed to run on Google Colab. No external dataset needed.

Required packages:
    pip install pandas numpy statsmodels scipy matplotlib
============================================================================
"""

# SETUP

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm

np.set_printoptions(precision=4)

print("=" * 60)
print("CHAPTER 2: MONTE CARLO SIMULATION METHODS")
print("=" * 60)

# 1. PSEUDO-RANDOM NUMBER GENERATORS

print("\n--- 1. Pseudo-Random Number Generators ---")
```

```
# Stata: set seed 10101; scalar u = runiform(); display u
np.random.seed(10101)
u = np.random.uniform()
print(f"Single uniform draw: {u:.6f}")

# Stata: set obs 10000; generate x = runiform()
np.random.seed(10101)
x = np.random.uniform(size=10000)
print(f"\n10,000 uniform draws:")
print(f"  Mean: {x.mean():.4f} (expected: 0.5)")
print(f"  Std:  {x.std():.4f} (expected: {1/np.sqrt(12):.4f})")
print(f"  Min:  {x.min():.4f}")
print(f"  Max:  {x.max():.4f}")

# Check independence: autocorrelations
print("\n--- Autocorrelations (independence check) ---")
for lag in [1, 2, 3]:
    r = np.corrcoef(x[lag:], x[:-lag])[0, 1]
    print(f"  Lag {lag}: r = {r:.6f} (should be ~0)")

# 2. GENERATING FROM VARIOUS DISTRIBUTIONS

print("\n--- 2. Multiple Distributions ---")

np.random.seed(10101)
n = 2000

# Stata equivalents
xt = stats.t.rvs(df=10, size=n)           # t(10)
xc = stats.chi2.rvs(df=10, size=n)        # Chi-squared(10)
xfn = stats.chi2.rvs(df=10, size=n) / 10  # Numerator of F(10,5)
xfd = stats.chi2.rvs(df=5, size=n) / 5    # Denominator of F(10,5)
xf = xfn / xfd                             # F(10,5)

results = pd.DataFrame({
    't(10)': xt, 'chi2(10)': xc, 'F(10,5)': xf
})
print(results.describe().round(3))

# Histogram + KDE
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, col, color in zip(axes, results.columns, ['#4472C4', '#ED7D31', '#70AD47']):
    results[col].hist(bins=50, ax=ax, density=True, alpha=0.7, color=color,
edgecolor='white')
    results[col].plot.kde(ax=ax, linewidth=2, color='black')
    ax.set_title(col, fontweight='bold')
plt.suptitle('Random Draws from t, Chi-squared, and F distributions', fontweight='bold')
plt.tight_layout()
plt.savefig('fig_ch2_distributions.png', dpi=150)
plt.show()

# 3. DISCRETE RANDOM VARIABLES

print("\n--- 3. Discrete Random Variables ---")

np.random.seed(10101)
```

```
# Poisson draws
xp = np.random.poisson(lam=5, size=n)

# Negative Binomial (Poisson-Gamma mixture)
xb = 4 + 2 * np.random.uniform(size=n)          # mu varies between 4 and 6
xg = np.random.gamma(shape=1, scale=1, size=n)   # gamma heterogeneity
xbh = xb * xg                                     # multiplicative heterogeneity
xp1 = np.random.poisson(lam=xb)                  # Poisson with varying mean
xp2 = np.random.poisson(lam=xbh)                 # NegBin (Poisson-Gamma)

disc_df = pd.DataFrame({'Poisson(5)': xp, 'Poisson(xb)': xp1, 'NB(xb)': xp2})
print(disc_df.describe().round(3))
print(f"\nOverdispersion check for NB(xb):")
print(f"  Var/Mean ratio = {xp2.var()/xp2.mean():.2f} (>1 means overdispersion)")

# 4. CENTRAL LIMIT THEOREM (CLT)

print("\n" + "=" * 60)
print("4. CENTRAL LIMIT THEOREM")
print("=" * 60)

# Stata: Draw 1 sample of size 30 from uniform
np.random.seed(10101)
one_sample = np.random.uniform(size=30)
print(f"\nOne sample (n=30) from Uniform(0,1):")
print(f"  Mean = {one_sample.mean():.4f}")
print(f"  Std  = {one_sample.std():.4f}")

# Stata: simulate 10,000 sample means
np.random.seed(10101)
n_reps = 10000
sample_size = 30
xbars = np.array([np.random.uniform(size=sample_size).mean() for _ in range(n_reps)])

print(f"\nDistribution of 10,000 sample means:")
print(f"  Mean of xbar:    {xbars.mean():.4f} (expected: 0.5)")
print(f"  Std of xbar:     {xbars.std():.4f} (expected: {1/np.sqrt(12*30):.4f})")
print(f"  Skewness:        {stats.skew(xbars):.4f} (expected: ~0)")
print(f"  Kurtosis (exc):  {stats.kurtosis(xbars):.4f} (expected: ~0)")

# Plot CLT demonstration
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(one_sample, bins=10, edgecolor='white', alpha=0.8, color='#4472C4')
axes[0].set_title('One Sample (n=30) from Uniform(0,1)', fontweight='bold')
axes[0].set_xlabel('x')

axes[1].hist(xbars, bins=50, density=True, edgecolor='white', alpha=0.8, color='#70AD47')
xg = np.linspace(xbars.min(), xbars.max(), 200)
mu, sigma = 0.5, 1 / np.sqrt(12 * 30)
axes[1].plot(xg, stats.norm.pdf(xg, mu, sigma), 'r-', linewidth=2, label=f'N({mu},
{sigma:.4f}²)')
axes[1].set_title('Distribution of 10,000 Sample Means (CLT)', fontweight='bold')
axes[1].set_xlabel(r'$\bar{x}$')
axes[1].legend()
```

```
plt.tight_layout()
plt.savefig('fig_ch2_clt.png', dpi=150)
plt.show()

# 5. FINITE-SAMPLE PROPERTIES OF OLS

print("\n" + "=" * 60)
print("5. SIMULATION: FINITE-SAMPLE PROPERTIES OF OLS")
print("=" * 60)

# DGP: y = 1 + 2*x + u, where u = chi2(1) - 1 (demeaned)
numobs = 150
numsims = 1000

np.random.seed(10101)

b2_list = []
se2_list = []
reject_list = []

for sim in range(numsims):
    x_sim = np.random.chisquare(df=1, size=numobs)
    u_sim = np.random.chisquare(df=1, size=numobs) - 1  # demeaned
    y_sim = 1 + 2 * x_sim + u_sim

    X_sim = sm.add_constant(x_sim)
    model_sim = sm.OLS(y_sim, X_sim).fit()

    b2 = model_sim.params[1]
    se2 = model_sim.bse[1]
    t2 = (b2 - 2) / se2  # t-statistic under H0: beta=2
    reject = abs(t2) > stats.t.ppf(0.975, numobs - 2)

    b2_list.append(b2)
    se2_list.append(se2)
    reject_list.append(reject)

b2_arr = np.array(b2_list)
se2_arr = np.array(se2_list)
reject_arr = np.array(reject_list)

print(f"\nResults from {numsims} simulations (N={numobs}):")
print(f"  True beta:          2.000")
print(f"  E[beta_hat]:        {b2_arr.mean():.4f}  (unbiased if ~2.0)")
print(f"  SD(beta_hat):       {b2_arr.std():.4f}")
print(f"  E[SE(beta_hat)]:    {se2_arr.mean():.4f}  (should be ~SD)")
print(f"  Rejection rate (5%): {reject_arr.mean():.4f}  (should be ~0.05)")

# Plot sampling distribution
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(b2_arr, bins=40, density=True, alpha=0.8, color='#ED7D31', edgecolor='white')
axes[0].axvline(x=2, color='red', linestyle='--', linewidth=2, label='True beta=2')
axes[0].set_title('Sampling Distribution of OLS beta_hat', fontweight='bold')
axes[0].set_xlabel(r'$\hat{\beta}$')
axes[0].legend()
```

```
t2_arr = (b2_arr - 2) / se2_arr
axes[1].hist(t2_arr, bins=40, density=True, alpha=0.8, color='#4472C4', edgecolor='white')
xg = np.linspace(-4, 4, 200)
axes[1].plot(xg, stats.t.pdf(xg, numobs - 2), 'r-', linewidth=2, label=f't({numobs-2})')
axes[1].set_title('t-Statistic vs Theoretical t-distribution', fontweight='bold')
axes[1].set_xlabel('t-statistic')
axes[1].legend()

plt.tight_layout()

plt.savefig('fig_ch2_ols_sim.png', dpi=150)
plt.show()

# 6. MEASUREMENT ERROR (ERRORS-IN-VARIABLES)

print("\n" + "=" * 60)
print("6. MEASUREMENT ERROR (ATTENUATION BIAS)")
print("=" * 60)

np.random.seed(10101)
n_meas = 10000

# True model: y = 1*xstar + u
# Observed: x = xstar + v (measurement error)
xstar = np.random.normal(0, 3, n_meas)  # Var(xstar) = 9
u = np.random.normal(0, 1, n_meas)       # Var(u) = 1
v = np.random.normal(0, 1, n_meas)       # Var(v) = 1

y_me = 1 * xstar + u                     # True DGP
x_me = xstar + v                         # Mismeasured x

# OLS with mismeasured x
model_me = sm.OLS(y_me, x_me).fit()

print(f"  True beta:    1.000")
print(f"  Estimated:    {model_me.params[0]:.4f}")
print(f"  Expected:     {9 / (9 + 1):.4f}  [= Var(x*)/(Var(x*)+Var(v))]")
print(f"  Attenuation:  {(1 - model_me.params[0]) * 100:.1f}% bias toward zero")

# 7. ENDOGENOUS REGRESSOR

print("\n" + "=" * 60)
print("7. ENDOGENOUS REGRESSOR")
print("=" * 60)

np.random.seed(10101)
n_endo_sims = 1000

b_endo_list = []
for _ in range(n_endo_sims):
    u_e = np.random.normal(size=numobs)
    z_e = np.random.normal(size=numobs)
    x_e = 0.5 * u_e + z_e        # x is correlated with u (endogenous!)
    y_e = 10 + 2 * x_e + u_e

    # OLS regression of y on x (biased!)
```

```
    model_e = sm.OLS(y_e, sm.add_constant(x_e)).fit()
    b_endo_list.append(model_e.params[1])

b_endo_arr = np.array(b_endo_list)
print(f"  True beta:     2.000")
print(f"  E[beta_OLS]:   {b_endo_arr.mean():.4f}  (biased upward!)")
print(f"  Bias:          {b_endo_arr.mean() - 2:.4f}")
print(f"  Expected bias: {0.5 * 1 / (0.5**2 * 1 + 1):.4f}  [= Cov(x,u)/Var(x)]")
print(f"\n  OLS is INCONSISTENT with endogenous regressors!")
print(f"  Solution: Use Instrumental Variables (Chapter 4)")

print("\n" + "=" * 60)
print("END OF CHAPTER 2")
print("=" * 60)
```

Appendix E: Python Code for Chapter 3 - Heteroskedasticity, SUR, and Survey Data

This appendix contains the complete Python code that replicates the Stata analysis from Chapter 3. The code is designed to run on Google Colab.

Setup Information

Dataset: Chap3_surdata.xlsx + Chap3_nhanes2.xlsx
Description: SUR data + NHANES II survey data
Required Python packages: pandas, numpy, statsmodels, scipy, matplotlib

To run on Google Colab:
1. Upload the Excel file(s) to your Colab session
2. Install packages: !pip install pandas, numpy, statsmodels, scipy, matplotlib
3. Run the code cells below

Chapter 3 Python Code

```
"""
============================================================================
EE627 Microeconometrics - Chapter 3
Heteroskedasticity, SUR, and Survey Data
============================================================================
Instructor: Asst. Prof. Dr. Tiraphap Fakthong
Thammasat University

Datasets: Chap3_surdata.xlsx, Chap3_nhanes2.xlsx
This script replicates "Chap 3 dofile.do" in Python.
Designed to run on Google Colab.

Required packages:
    pip install pandas numpy statsmodels scipy matplotlib openpyxl
============================================================================
"""

# SETUP

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.optimize import minimize
import statsmodels.api as sm
from statsmodels.stats.diagnostic import het_breuschpagan

np.set_printoptions(precision=4)

print("=" * 60)
print("CHAPTER 3: HETEROSKEDASTICITY, SUR, AND SURVEY DATA")
print("=" * 60)

# PART 1: MODELING HETEROSKEDASTIC DATA (Generated Data)
```

```
print("\n" + "=" * 60)
print("PART 1: HETEROSKEDASTICITY - FGLS AND WLS")
print("=" * 60)

# Generate data matching the Stata DGP:
# y = 1 + 1*x2 + 1*x3 + u
# u = sqrt(exp(-1 + 0.2*x2)) * e
# Var(u|x) = exp(-1 + 0.2*x2)  -- heteroskedastic!
np.random.seed(10101)
n = 500
x2 = 5 * np.random.normal(size=n)
x3 = 5 * np.random.normal(size=n)
e = 5 * np.random.normal(size=n)
u = np.sqrt(np.exp(-1 + 0.2 * x2)) * e
y = 1 + 1 * x2 + 1 * x3 + u

df_het = pd.DataFrame({'y': y, 'x2': x2, 'x3': x3})
print("\nGenerated data summary:")
print(df_het.describe().round(3))

# --- OLS with default SEs ---
X = sm.add_constant(df_het[['x2', 'x3']])
model_default = sm.OLS(y, X).fit()
print("\n--- OLS with Default SEs (INCORRECT for inference) ---")
print(model_default.summary2().tables[1].round(4))

# --- OLS with robust SEs ---
model_robust = sm.OLS(y, X).fit(cov_type='HC1')
print("\n--- OLS with Robust SEs (HC1) ---")
print(model_robust.summary2().tables[1].round(4))

# --- Heteroskedasticity test ---
print("\n--- Breusch-Pagan Test ---")
bp_stat, bp_pval, _, _ = het_breuschpagan(model_default.resid, X)
print(f"  LM stat = {bp_stat:.4f}, p-value = {bp_pval:.6f}")
print(f"  Result: {'REJECT H0 (heteroskedastic)' if bp_pval < 0.05 else 'Fail to reject
H0'}")

# --- Heteroskedasticity diagnostic plot ---
uhat = model_default.resid
absu = np.abs(uhat)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].scatter(x2, absu, alpha=0.3, s=10, color='#4472C4')
z = np.polyfit(x2, absu, 3)
p_fit = np.poly1d(z)
x2_sorted = np.sort(x2)
axes[0].plot(x2_sorted, p_fit(x2_sorted), 'r-', linewidth=2, label='Polynomial fit')
axes[0].set_xlabel('x2')
axes[0].set_ylabel('|residual|')
axes[0].set_title('|Residual| vs x2 (Increasing variance)', fontweight='bold')
axes[0].legend()

axes[1].scatter(x3, absu, alpha=0.3, s=10, color='#ED7D31')
z3 = np.polyfit(x3, absu, 3)
p_fit3 = np.poly1d(z3)
```

```
x3_sorted = np.sort(x3)
axes[1].plot(x3_sorted, p_fit3(x3_sorted), 'r-', linewidth=2, label='Polynomial fit')
axes[1].set_xlabel('x3')
axes[1].set_ylabel('|residual|')
axes[1].set_title('|Residual| vs x3 (Constant variance)', fontweight='bold')

axes[1].legend()

plt.tight_layout()
plt.savefig('fig_ch3_hetdiag.png', dpi=150)
plt.show()

# --- FGLS Estimation ---
print("\n--- FGLS (Feasible GLS) ---")

# Step 1: Get OLS residuals
uhat_sq = uhat ** 2

# Step 2: Model variance function: E[u^2|x] = exp(alpha0 + alpha1*x2)
# Using NLS (nonlinear least squares) via OLS on log(uhat^2)
log_uhat_sq = np.log(uhat_sq + 1e-10)  # add small constant to avoid log(0)
Z = sm.add_constant(x2)
var_model = sm.OLS(log_uhat_sq, Z).fit()
print(f"  Variance model: log(u^2) = {var_model.params[0]:.3f} +
{var_model.params[1]:.3f}*x2")
print(f"  (True: -1 + 0.2*x2)")

# Step 3: Estimated variance
varu = np.exp(var_model.fittedvalues)

# Step 4: FGLS (WLS with default SEs)
model_fgls = sm.WLS(y, X, weights=1.0 / varu).fit()
print(f"\n  FGLS estimates:")
print(model_fgls.summary2().tables[1].round(4))

# Step 5: WLS = FGLS + robust SEs (RECOMMENDED)
model_wls = sm.WLS(y, X, weights=1.0 / varu).fit(cov_type='HC1')
print(f"\n--- WLS (FGLS + Robust SEs) - RECOMMENDED ---")
print(model_wls.summary2().tables[1].round(4))

# --- Comparison table ---
print("\n--- Comparison of Estimators ---")
comp = pd.DataFrame({
    'OLS_default': model_default.params.round(4),
    'SE_default': model_default.bse.round(4),
    'OLS_robust': model_robust.params.round(4),
    'SE_robust': model_robust.bse.round(4),
    'FGLS': model_fgls.params.round(4),
    'SE_fgls': model_fgls.bse.round(4),
    'WLS': model_wls.params.round(4),
    'SE_wls': model_wls.bse.round(4),
})
print(comp)

# PART 2: SEEMINGLY UNRELATED REGRESSIONS (SUR)
```

```
print("\n" + "=" * 60)
print("PART 2: SEEMINGLY UNRELATED REGRESSIONS (SUR)")
print("=" * 60)

df_sur = pd.read_excel("Chap3_surdata.xlsx")
print("\nSUR data summary:")
sur_vars = ['ldrugexp', 'ltotothr', 'age', 'age2', 'educyr', 'actlim', 'totchr', 'medicaid',
'private']
print(df_sur[sur_vars].describe().round(3))

# Note: Python's statsmodels does not have a direct SUR command.
# We use equation-by-equation OLS as a starting point.
# For true SUR, use the systemfit or linearmodels package.

# Equation 1: ldrugexp = f(age, age2, actlim, totchr, medicaid, private)
# Equation 2: ltotothr = f(age, age2, educyr, actlim, totchr, private)

df_sur_clean = df_sur[['ldrugexp', 'ltotothr', 'age', 'age2', 'educyr',
                       'actlim', 'totchr', 'medicaid', 'private']].dropna()

print(f"\nUsable observations (non-missing in both equations): {len(df_sur_clean)}")

# Equation 1
X1 = sm.add_constant(df_sur_clean[['age', 'age2', 'actlim', 'totchr', 'medicaid',
'private']])
y1 = df_sur_clean['ldrugexp']
eq1 = sm.OLS(y1, X1).fit(cov_type='HC1')

# Equation 2
X2 = sm.add_constant(df_sur_clean[['age', 'age2', 'educyr', 'actlim', 'totchr', 'private']])
y2 = df_sur_clean['ltotothr']
eq2 = sm.OLS(y2, X2).fit(cov_type='HC1')

print("\n--- Equation 1: Log Drug Expenditure ---")
print(eq1.summary2().tables[1].round(4))

print("\n--- Equation 2: Log Other Medical Expenditure ---")
print(eq2.summary2().tables[1].round(4))

# Cross-equation residual correlation
resid_corr = np.corrcoef(eq1.resid[:len(eq2.resid)], eq2.resid[:len(eq1.resid)])[0, 1]
print(f"\n  Residual correlation between equations: {resid_corr:.4f}")
print(f"  (If this is high, SUR gains efficiency over equation-by-equation OLS)")

# Try SUR using linearmodels if available
try:
    from linearmodels.system import SUR as SystemSUR
    print("\n--- SUR Estimation (using linearmodels) ---")
    sur_data = {
        'eq1': {'dependent': y1, 'exog': X1},
        'eq2': {'dependent': y2, 'exog': X2},
    }
    # Using formula interface
    import linearmodels
    # Direct SUR from data
    from linearmodels.system import SUR as SystemSUR
```

```
    print("  SUR estimation available via linearmodels.system.SUR")
    print("  For full SUR implementation, see the companion Colab notebook.")
except ImportError:
    print("\n  Note: For full SUR estimation, install linearmodels:")
    print("  pip install linearmodels")

# PART 3: SURVEY DATA (NHANES II)

print("\n" + "=" * 60)
print("PART 3: SURVEY DATA (NHANES II)")
print("=" * 60)

df_nhanes = pd.read_excel("Chap3_nhanes2.xlsx")
df_nhanes = df_nhanes[(df_nhanes['age'] >= 21) & (df_nhanes['age'] <= 65)]

print(f"\nNHANES II data: {len(df_nhanes)} observations (age 21-65)")
print(f"\nSurvey design variables:")
for v in ['sampl', 'finalwgt', 'strata', 'psu']:
    if v in df_nhanes.columns:
        print(f"  {v}: {df_nhanes[v].describe().round(2).to_dict()}")

# --- Unweighted regression ---
print("\n--- Unweighted OLS: hgb on age, female ---")
nhanes_vars = ['hgb', 'age', 'female']
df_nh = df_nhanes[nhanes_vars + ['finalwgt', 'strata', 'psu']].dropna()

X_nh = sm.add_constant(df_nh[['age', 'female']])
y_nh = df_nh['hgb']
model_unwt = sm.OLS(y_nh, X_nh).fit()
print(model_unwt.summary2().tables[1].round(4))

# --- Weighted regression (pweight) ---
print("\n--- Weighted OLS (pweight = finalwgt) ---")
model_wt_nh = sm.WLS(y_nh, X_nh, weights=df_nh['finalwgt']).fit(cov_type='HC1')
print(model_wt_nh.summary2().tables[1].round(4))

# --- Weighted with cluster-robust SEs ---
print("\n--- Weighted OLS with Cluster-Robust SEs (cluster on PSU) ---")
# Create unique PSU identifier
df_nh['uniqpsu'] = 2 * df_nh['strata'] + df_nh['psu']
model_svyreg = sm.WLS(y_nh, X_nh, weights=df_nh['finalwgt']).fit(
    cov_type='cluster', cov_kwds={'groups': df_nh['uniqpsu']}
)
print(model_svyreg.summary2().tables[1].round(4))

# --- Population mean ---
print("\n--- Population Mean of hgb ---")
hgb = df_nh['hgb']
wt = df_nh['finalwgt']
wtd_mean = np.average(hgb, weights=wt)
unwt_mean = hgb.mean()
print(f"  Unweighted mean: {unwt_mean:.4f}")
print(f"  Weighted mean:   {wtd_mean:.4f}")

print("\n  Note: Differences between weighted and unweighted estimates")
print("  indicate that the sampling design is not self-weighting.")
```

```
print("  Use svy: prefix in Stata, or WLS with pweights in Python.")

# --- Comparison ---
print("\n--- Comparison: Unweighted vs Weighted vs Survey ---")
comp_survey = pd.DataFrame({
    'Unweighted': model_unwt.params.round(4),
    'SE_unwt': model_unwt.bse.round(4),
    'Weighted': model_wt_nh.params.round(4),
    'SE_wt': model_wt_nh.bse.round(4),
    'Survey': model_svyreg.params.round(4),
    'SE_survey': model_svyreg.bse.round(4),
})

print(comp_survey)

print("\n" + "=" * 60)
print("END OF CHAPTER 3")
print("=" * 60)
```

Appendix F: Python Code for Chapter 4 - Instrumental Variables Estimation

This appendix contains the complete Python code that replicates the Stata analysis from Chapter 4. The code is designed to run on Google Colab.

Setup Information

Dataset: Chap4_data.xlsx
Description: Drug expenditure and insurance data
Required Python packages: pandas, numpy, statsmodels, scipy, matplotlib, linearmodels

To run on Google Colab:
1. Upload the Excel file(s) to your Colab session
2. Install packages: !pip install pandas, numpy, statsmodels, scipy, matplotlib, linearmodels
3. Run the code cells below

Chapter 4 Python Code

```
"""
============================================================================
EE627 Microeconometrics - Chapter 4
Instrumental Variables Estimation
============================================================================
Instructor: Asst. Prof. Dr. Tiraphap Fakthong
Thammasat University

Dataset: Chap4_data.xlsx
This script replicates "Chap 4.do" in Python.
Designed to run on Google Colab.

Required packages:
    pip install pandas numpy statsmodels scipy matplotlib linearmodels openpyxl
============================================================================
"""

# SETUP

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
try:
    from linearmodels.iv import IV2SLS, IVGMM, IVLIML
except ImportError:
    raise ImportError(
        "This script requires the 'linearmodels' package.\n"
        "Install with: pip install linearmodels"
    )

np.set_printoptions(precision=4)

print("=" * 60)
```

```
print("CHAPTER 4: INSTRUMENTAL VARIABLES ESTIMATION")
print("=" * 60)

# 1. LOAD DATA

df = pd.read_excel("Chap4_data.xlsx")

x2list = ['totchr', 'age', 'female', 'blhisp', 'linc']
dep_var = 'ldrugexp'
endog_var = 'hi_empunion'
instruments = ['ssiratio', 'lowincome', 'multlc', 'firmsz']

# Keep complete cases
all_vars = [dep_var, endog_var] + x2list + instruments
df_iv = df[all_vars].dropna()

print(f"\nSample size: {len(df_iv)}")
print(f"\nSummary of key variables:")
print(df_iv[[dep_var, endog_var] + instruments].describe().round(3))

# 2. OLS ESTIMATION (for comparison)

print("\n" + "=" * 60)
print("2. OLS ESTIMATION (Baseline)")
print("=" * 60)

X_ols = sm.add_constant(df_iv[[endog_var] + x2list])
y = df_iv[dep_var]
model_ols = sm.OLS(y, X_ols).fit(cov_type='HC1')
print(model_ols.summary2().tables[1].round(4))

# 3. IV ESTIMATION: 2SLS (Just-Identified)

print("\n" + "=" * 60)
print("3. 2SLS - JUST-IDENTIFIED (1 instrument: ssiratio)")
print("=" * 60)

# Stata: ivregress 2sls ldrugexp (hi_empunion = ssiratio) $x2list, vce(robust) first

# Using linearmodels
y_iv = df_iv[dep_var]
endog = df_iv[[endog_var]]
exog = sm.add_constant(df_iv[x2list])
instruments_just = df_iv[['ssiratio']]

model_2sls_just = IV2SLS(y_iv, exog, endog, instruments_just).fit(cov_type='robust')
print(model_2sls_just.summary)

# 4. FIRST STAGE

print("\n" + "=" * 60)
print("4. FIRST STAGE REGRESSION")
print("=" * 60)

X_first = sm.add_constant(df_iv[['ssiratio'] + x2list])
model_first = sm.OLS(df_iv[endog_var], X_first).fit(cov_type='HC1')
print(model_first.summary2().tables[1].round(4))
```

```
print(f"\n  First-stage F-statistic for ssiratio: {model_first.fvalue:.2f}")
print(f"  Rule of thumb: F > 10 means instruments are strong")
print(f"  Result: {'STRONG' if model_first.fvalue > 10 else 'WEAK'} instrument")

# 5. IV ESTIMATION: 2SLS (Over-Identified)

print("\n" + "=" * 60)
print("5. 2SLS - OVER-IDENTIFIED (2 instruments: ssiratio, multlc)")
print("=" * 60)

instruments_over = df_iv[['ssiratio', 'multlc']]
model_2sls_over = IV2SLS(y_iv, exog, endog, instruments_over).fit(cov_type='robust')
print(model_2sls_over.summary)

# 6. GMM ESTIMATION

print("\n" + "=" * 60)
print("6. GMM ESTIMATION (Efficient under heteroskedasticity)")
print("=" * 60)

model_gmm = IVGMM(y_iv, exog, endog, instruments_over).fit(cov_type='robust')
print(model_gmm.summary)

# 7. LIML ESTIMATION

print("\n" + "=" * 60)
print("7. LIML ESTIMATION (Less biased with weak instruments)")
print("=" * 60)

instruments_all = df_iv[['ssiratio', 'lowincome', 'multlc', 'firmsz']]
model_liml = IVLIML(y_iv, exog, endog, instruments_all).fit(cov_type='robust')
print(model_liml.summary)

# 8. COMPARISON OF ESTIMATORS

print("\n" + "=" * 60)
print("8. COMPARISON OF ESTIMATORS")
print("=" * 60)

comparison = pd.DataFrame({
    'OLS': model_ols.params[[endog_var] + x2list].round(4),
    '2SLS_just': [model_2sls_just.params[endog_var]] + [model_2sls_just.params[v] for v in
x2list],
    '2SLS_over': [model_2sls_over.params[endog_var]] + [model_2sls_over.params[v] for v in
x2list],
    'GMM': [model_gmm.params[endog_var]] + [model_gmm.params[v] for v in x2list],
    'LIML': [model_liml.params[endog_var]] + [model_liml.params[v] for v in x2list],
}, index=[endog_var] + x2list)

print(comparison.round(4))

# 9. ENDOGENEITY TEST (Durbin-Wu-Hausman)

print("\n" + "=" * 60)
print("9. ENDOGENEITY TEST (Durbin-Wu-Hausman)")
print("=" * 60)

# Manual DWH test:
```

```
# Step 1: Regress endogenous variable on instruments + exogenous
X_first_stage = sm.add_constant(df_iv[['ssiratio'] + x2list])
first_stage = sm.OLS(df_iv[endog_var], X_first_stage).fit()
v1hat = first_stage.resid

# Step 2: Include residual in structural equation
X_aug = sm.add_constant(df_iv[[endog_var] + x2list])
X_aug = X_aug.copy()
X_aug['v1hat'] = v1hat.values
model_aug = sm.OLS(y, X_aug).fit(cov_type='HC1')

t_v1hat = model_aug.tvalues['v1hat']
p_v1hat = model_aug.pvalues['v1hat']

print(f"  t-statistic on v1hat: {t_v1hat:.4f}")
print(f"  p-value:              {p_v1hat:.4f}")
if p_v1hat < 0.05:
    print("  -> REJECT H0: hi_empunion IS endogenous -> Use IV")
else:
    print("  -> FAIL TO REJECT H0: No evidence of endogeneity -> OLS may be OK")

# 10. OVERIDENTIFICATION TEST (Hansen J)

print("\n" + "=" * 60)
print("10. OVERIDENTIFICATION TEST")
print("=" * 60)

# The GMM model from linearmodels provides the J-test
print(f"  Hansen J-statistic: {model_gmm.j_stat.stat:.4f}")
print(f"  p-value:            {model_gmm.j_stat.pval:.4f}")
if model_gmm.j_stat.pval > 0.05:
    print("  -> FAIL TO REJECT H0: Instruments are valid (overidentifying restrictions
hold)")
else:
    print("  -> REJECT H0: At least one instrument may be invalid")

# 11. WEAK INSTRUMENTS

print("\n" + "=" * 60)
print("11. WEAK INSTRUMENT DIAGNOSTICS")
print("=" * 60)

print("\n--- Correlations between endogenous variable and instruments ---")
corr_iv = df_iv[[endog_var] + instruments].corr()[endog_var].drop(endog_var)
print(corr_iv.round(4))

# First-stage F for each instrument separately
print("\n--- First-Stage F-statistics (individual instruments) ---")
for inst in instruments:
    X_fs = sm.add_constant(df_iv[[inst] + x2list])
    fs_model = sm.OLS(df_iv[endog_var], X_fs).fit()
    # Partial F-test for the excluded instrument
    r_model = sm.OLS(df_iv[endog_var], sm.add_constant(df_iv[x2list])).fit()
    f_stat = ((r_model.ssr - fs_model.ssr) / 1) / (fs_model.ssr / fs_model.df_resid)
    print(f"  {inst:12s}: F = {f_stat:.2f}  {'STRONG' if f_stat > 10 else 'WEAK'}")

# Compare IV estimates with different instruments
```

```
print("\n--- IV Estimates with Different Single Instruments ---")
for i, inst in enumerate(instruments):
    try:
        model_i = IV2SLS(y_iv, exog, endog, df_iv[[inst]]).fit(cov_type='robust')
        print(f"  {inst:12s}: beta(hi_empunion) = {model_i.params[endog_var]:.4f}, "
              f"SE = {model_i.std_errors[endog_var]:.4f}")
    except Exception as e:
        print(f"  {inst:12s}: Error - {e}")

print("\n  Note: If estimates differ widely across instruments, this suggests")
print("  some instruments may be weak or invalid.")

print("\n" + "=" * 60)
print("END OF CHAPTER 4")
print("=" * 60)
```

Appendix G: Python Code for Chapter 5 - Quantile Regression and Count Data Models

This appendix contains the complete Python code that replicates the Stata analysis from Chapter 5. The code is designed to run on Google Colab.

Setup Information

Dataset: Chap5_1_data.xlsx + Chap5_2_data.xlsx
Description: Medical expenditures + Doctor visits
Required Python packages: pandas, numpy, statsmodels, scipy, matplotlib

To run on Google Colab:
1. Upload the Excel file(s) to your Colab session
2. Install packages: !pip install pandas, numpy, statsmodels, scipy, matplotlib
3. Run the code cells below

Chapter 5 Python Code

```
"""
============================================================================
EE627 Microeconometrics - Chapter 5
Quantile Regression and Count Data Models
============================================================================
Instructor: Asst. Prof. Dr. Tiraphap Fakthong
Thammasat University

Datasets: Chap5_1_data.xlsx (medical expenditures), Chap5_2_data.xlsx (doctor visits)
This script replicates "Chap 5.do" in Python.
Designed to run on Google Colab.

Required packages:
    pip install pandas numpy statsmodels scipy matplotlib openpyxl
============================================================================
"""

# SETUP

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.regression.quantile_regression import QuantReg

np.set_printoptions(precision=4)

print("=" * 60)
print("CHAPTER 5: QUANTILE REGRESSION & COUNT DATA MODELS")
print("=" * 60)

# PART 1: QUANTILE REGRESSION FOR MEDICAL EXPENDITURES
```

```
print("\n" + "=" * 60)
print("PART 1: QUANTILE REGRESSION")
print("=" * 60)

# Load data
df = pd.read_excel("Chap5_1_data.xlsx")
df = df.dropna(subset=['ltotexp'])
print(f"\nSample size: {len(df)}")

key_vars = ['ltotexp', 'suppins', 'totchr', 'age', 'female', 'white']
print("\n--- Summary Statistics ---")
print(df[key_vars].describe().round(3))

# 1. OLS (for comparison)

print("\n--- OLS Regression (Conditional Mean) ---")
X = sm.add_constant(df[['suppins', 'totchr', 'age', 'female', 'white']])
y = df['ltotexp']
model_ols = sm.OLS(y, X).fit(cov_type='HC1')
print(model_ols.summary2().tables[1].round(4))

# 2. QUANTILE REGRESSION AT DIFFERENT QUANTILES

quantiles = [0.25, 0.50, 0.75]
qr_models = {}

for q in quantiles:
    print(f"\n--- Quantile Regression: q = {q} ---")
    model_qr = QuantReg(y, X).fit(q=q)
    qr_models[q] = model_qr
    print(model_qr.summary().tables[1])

# 3. COMPARISON TABLE: OLS vs QR

print("\n" + "=" * 60)
print("COMPARISON: OLS vs Quantile Regression")
print("=" * 60)

coef_names = ['const', 'suppins', 'totchr', 'age', 'female', 'white']
comparison = pd.DataFrame({'OLS': model_ols.params.round(4)})
for q in quantiles:
    comparison[f'QR({q})'] = qr_models[q].params.round(4)
comparison.index = coef_names
print(comparison)

# 4. PLOT: HOW COEFFICIENTS CHANGE ACROSS QUANTILES

print("\n--- Plotting QR coefficients across quantiles ---")

quantile_range = np.arange(0.05, 0.96, 0.05)
coef_results = {var: [] for var in coef_names}
ci_lower = {var: [] for var in coef_names}
ci_upper = {var: [] for var in coef_names}

for q in quantile_range:
    model_q = QuantReg(y, X).fit(q=q)
    for i, var in enumerate(coef_names):
```

```
        coef_results[var].append(model_q.params.iloc[i])
        ci_lower[var].append(model_q.conf_int().iloc[i, 0])
        ci_upper[var].append(model_q.conf_int().iloc[i, 1])

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.flatten()

for idx, var in enumerate(coef_names):
    ax = axes[idx]
    ax.plot(quantile_range, coef_results[var], 'b-', linewidth=2, label='QR')
    ax.fill_between(quantile_range, ci_lower[var], ci_upper[var], alpha=0.2, color='blue')
    ax.axhline(y=model_ols.params.iloc[idx], color='red', linestyle='--', linewidth=1.5,
label='OLS')
    ax.set_title(var, fontweight='bold')
    ax.set_xlabel('Quantile')
    if idx == 0:
        ax.legend(fontsize=8)

plt.suptitle('Quantile Regression Coefficients Across Quantiles\n(with 95% CI, OLS in red)',
             fontweight='bold', fontsize=13)
plt.tight_layout()
plt.savefig('fig_ch5_qrcoef.png', dpi=150)
plt.show()
print("Figure saved: fig_ch5_qrcoef.png")

# 5. TEST EQUALITY ACROSS QUANTILES

print("\n" + "=" * 60)
print("TEST: Are coefficients equal across quantiles?")
print("=" * 60)

# Manual Wald-type test comparing q=0.25 and q=0.75
# For suppins:
b25 = qr_models[0.25].params['suppins']
b75 = qr_models[0.75].params['suppins']
se25 = qr_models[0.25].bse['suppins']
se75 = qr_models[0.75].bse['suppins']
z_stat = (b25 - b75) / np.sqrt(se25**2 + se75**2)
p_val = 2 * (1 - stats.norm.cdf(abs(z_stat)))

print(f"\nTest: beta_suppins(q=0.25) = beta_suppins(q=0.75)")
print(f"  beta(0.25) = {b25:.4f}, beta(0.75) = {b75:.4f}")
print(f"  z-statistic = {z_stat:.4f}")
print(f"  p-value = {p_val:.4f}")
if p_val < 0.05:
    print("  -> REJECT: Coefficients differ across quantiles (heterogeneous effects!)")
else:
    print("  -> Fail to reject: No evidence of different effects across quantiles")

# 6. MULTIPLIER FOR CONVERTING LOG QR TO LEVEL AME

print("\n--- Converting QR in Logs to AME in Levels ---")
xb = qr_models[0.50].fittedvalues
expxb = np.exp(xb)
multiplier = expxb.mean()
print(f"  Multiplier = E[exp(x'beta)] = {multiplier:.2f}")
```

```
print(f"  AME of suppins in levels at q=0.50: {qr_models[0.50].params['suppins'] *
multiplier:.2f}")

# PART 2: QUANTILE REGRESSION FOR GENERATED HETEROSKEDASTIC DATA

print("\n" + "=" * 60)
print("PART 2: QR FOR GENERATED HETEROSKEDASTIC DATA")
print("=" * 60)

np.random.seed(10101)
n = 10000
x2 = np.random.chisquare(1, n)
x3 = 5 * np.random.randn(n)
e = 5 * np.random.randn(n)
u = (0.1 + 0.5 * x2) * e
y_gen = 1 + 1 * x2 + 1 * x3 + u

X_gen = sm.add_constant(np.column_stack([x2, x3]))

print("\n--- Quantile Regression for Generated Data ---")
gen_results = {}
for q in [0.25, 0.50, 0.75]:
    model_gen = QuantReg(y_gen, X_gen).fit(q=q)
    gen_results[q] = model_gen.params
    print(f"  q={q}: beta_x2 = {model_gen.params[1]:.4f}, beta_x3 =
{model_gen.params[2]:.4f}")

# Predicted coefficients
e_quantiles = np.quantile(e, [0.25, 0.50, 0.75])
print(f"\n  Predicted coefficient of x2:")
for q, eq in zip([0.25, 0.50, 0.75], e_quantiles):
    pred = 1 + 0.5 * eq
    print(f"    q={q}: predicted = {pred:.4f}, estimated = {gen_results[q][1]:.4f}")

# PART 3: COUNT DATA MODELS

print("\n" + "=" * 60)
print("PART 3: COUNT DATA (DOCTOR VISITS)")
print("=" * 60)

df_count = pd.read_excel("Chap5_2_data.xlsx")
print(f"\nSample size: {len(df_count)}")
count_vars = ['docvis', 'private', 'totchr', 'age', 'female', 'white']
print("\n--- Summary Statistics ---")
print(df_count[count_vars].describe().round(3))

# Overdispersion check
mean_y = df_count['docvis'].mean()
var_y = df_count['docvis'].var()
print(f"\n  Mean(docvis) = {mean_y:.3f}")
print(f"  Var(docvis)  = {var_y:.3f}")
print(f"  Var/Mean     = {var_y/mean_y:.3f} (>1 means overdispersion)")

# --- Poisson regression ---
print("\n--- Poisson Regression ---")
model_pois = sm.GLM(
    df_count['docvis'],
```

```
    sm.add_constant(df_count[['private', 'totchr', 'age', 'female', 'white']]),
    family=sm.families.Poisson()
).fit(cov_type='HC1')
print(model_pois.summary2().tables[1].round(4))

# --- Negative Binomial regression ---
print("\n--- Negative Binomial Regression ---")
model_nb = sm.GLM(
    df_count['docvis'],
    sm.add_constant(df_count[['private', 'totchr', 'age', 'female', 'white']]),
    family=sm.families.NegativeBinomial(alpha=1.0)
).fit(cov_type='HC1')
print(model_nb.summary2().tables[1].round(4))

# --- Marginal effects (at the mean) ---
print("\n--- Marginal Effects at the Mean (Poisson) ---")
X_mean = sm.add_constant(df_count[['private', 'totchr', 'age', 'female', 'white']]).mean()
mu_mean = np.exp(model_pois.params @ X_mean)
print(f"  Predicted count at means: {mu_mean:.3f}")
for var in ['private', 'totchr', 'age', 'female', 'white']:
    me = model_pois.params[var] * mu_mean
    print(f"  dE[Y]/d({var}) = {me:.4f}")

# --- Quantile count regression (jittered) ---
print("\n--- Quantile Regression on Jittered Counts ---")
np.random.seed(10101)
df_count['docvis_jitter'] = df_count['docvis'] + np.random.uniform(size=len(df_count))

X_count = sm.add_constant(df_count[['private', 'totchr', 'age', 'female', 'white']])
for q in [0.50, 0.75]:
    model_qc = QuantReg(df_count['docvis_jitter'], X_count).fit(q=q)
    print(f"\n  Quantile Count (jittered) at q={q}:")
    for var in ['private', 'totchr', 'age', 'female', 'white']:
        print(f"    {var}: {model_qc.params[var]:.4f}")

print("\n" + "=" * 60)
print("END OF CHAPTER 5")
print("=" * 60)
```

Appendix H: Python Code for Chapter 6 - Panel Data: Linear and Nonlinear Models

This appendix contains the complete Python code that replicates the Stata analysis from Chapter 6. The code is designed to run on Google Colab.

Setup Information

Dataset: Chap6_panel.xlsx + Chap6_npanel.xlsx
Description: PSID panel + RAND HIE data
Required Python packages: pandas, numpy, statsmodels, scipy, matplotlib, linearmodels

To run on Google Colab:
1. Upload the Excel file(s) to your Colab session
2. Install packages: !pip install pandas, numpy, statsmodels, scipy, matplotlib, linearmodels
3. Run the code cells below

Chapter 6 Python Code

```
"""
==============================================================================
EE627 Microeconometrics - Chapter 6
Panel Data: Linear and Nonlinear Models
==============================================================================
Instructor: Asst. Prof. Dr. Tiraphap Fakthong
Thammasat University

Datasets: Chap6_panel.xlsx (PSID), Chap6_npanel.xlsx (RAND HIE)
This script replicates "Chap 6 panel.do" and "Chap 6 npanel.do" in Python.
Designed to run on Google Colab.

Required packages:
    pip install pandas numpy statsmodels scipy matplotlib linearmodels openpyxl
==============================================================================
"""

# SETUP

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
try:
    from linearmodels.panel import (
        PooledOLS, BetweenOLS, FirstDifferenceOLS,
        PanelOLS, RandomEffects
    )
except ImportError:
    raise ImportError(
        "This script requires the 'linearmodels' package.\n"
        "Install with: pip install linearmodels"
    )
```

```
np.set_printoptions(precision=4)
pd.set_option('display.max_columns', 15)

print("=" * 60)
print("CHAPTER 6: PANEL DATA - LINEAR AND NONLINEAR MODELS")
print("=" * 60)

# PART A: LINEAR PANEL DATA (PSID)

print("\n" + "=" * 60)
print("PART A: LINEAR PANEL DATA (PSID)")
print("=" * 60)

df = pd.read_excel("Chap6_panel.xlsx")
print(f"\nDataset: 595 individuals x 7 years (1976-82)")
print(f"Total observations: {len(df)}")
print(f"\n--- Summary Statistics ---")
print(df[['lwage', 'exp', 'exp2', 'wks', 'ed', 'south']].describe().round(3))

# Set panel index
df = df.set_index(['id', 't'])

# 1. PANEL SUMMARY (xtsum equivalent)

print("\n--- Panel Summary: Within and Between Variation ---")
for var in ['lwage', 'exp', 'wks', 'ed']:
    overall_mean = df[var].mean()
    overall_std = df[var].std()
    between_std = df[var].groupby('id').mean().std()
    within_std = (df[var] - df[var].groupby('id').transform('mean')).std()
    print(f"  {var:8s}: overall SD={overall_std:.4f}, between SD={between_std:.4f}, within
SD={within_std:.4f}")

# 2. TIME-SERIES PLOTS

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df_plot = df.reset_index()
for i, uid in enumerate(df_plot['id'].unique()[:20]):
    sub = df_plot[df_plot['id'] == uid]
    axes[0].plot(sub['t'], sub['lwage'], alpha=0.6, linewidth=1)
    axes[1].plot(sub['t'], sub['wks'], alpha=0.6, linewidth=1)
axes[0].set_title('Log Wage by Individual', fontweight='bold')
axes[0].set_xlabel('Time'); axes[0].set_ylabel('lwage')
axes[1].set_title('Weeks Worked by Individual', fontweight='bold')
axes[1].set_xlabel('Time'); axes[1].set_ylabel('wks')
plt.tight_layout()
plt.savefig('fig_ch6_panel_ts.png', dpi=150)
plt.show()

# 3. POOLED OLS

print("\n--- Pooled OLS (with cluster-robust SEs) ---")
# Stata: regress lwage exp exp2 wks ed, vce(cluster id)
exog_vars = ['exp', 'exp2', 'wks', 'ed']
y = df['lwage']
X = sm.add_constant(df[exog_vars])
```

```
model_pooled = PooledOLS(y, X).fit(cov_type='clustered', cluster_entity=True)
print(model_pooled.summary.tables[1])

# 4. BETWEEN ESTIMATOR

print("\n--- Between Estimator ---")
# Stata: xtreg lwage exp exp2 wks ed, be
model_be = BetweenOLS(y, X).fit()
print(model_be.summary.tables[1])

# 5. FIXED EFFECTS (WITHIN) ESTIMATOR

print("\n--- Fixed Effects (Within) Estimator ---")
# Stata: xtreg lwage exp exp2 wks ed, fe vce(cluster id)
# Note: FE drops time-invariant variables (ed)
model_fe = PanelOLS(y, df[['exp', 'exp2', 'wks']], entity_effects=True).fit(
    cov_type='clustered', cluster_entity=True)
print(model_fe.summary.tables[1])

# FE with ed included (will be absorbed by entity effects)
print("\n  Note: In FE, time-invariant variable 'ed' is absorbed by individual effects")
print(f"  R-squared (within): {model_fe.rsquared_within:.4f}")
print(f"  R-squared (between): {model_fe.rsquared_between:.4f}")
print(f"  R-squared (overall): {model_fe.rsquared_overall:.4f}")

# 6. RANDOM EFFECTS ESTIMATOR

print("\n--- Random Effects Estimator ---")
# Stata: xtreg lwage exp exp2 wks ed, re vce(cluster id) theta
model_re = RandomEffects(y, X).fit(cov_type='clustered', cluster_entity=True)
print(model_re.summary.tables[1])

# 7. FIRST DIFFERENCES

print("\n--- First Differences Estimator ---")
# Stata: regress D.(lwage exp exp2 wks ed), vce(cluster id) noconstant
model_fd = FirstDifferenceOLS(y, df[['exp', 'exp2', 'wks']]).fit(
    cov_type='clustered', cluster_entity=True)
print(model_fd.summary.tables[1])

# 8. COMPARISON OF ALL ESTIMATORS

print("\n" + "=" * 60)
print("COMPARISON OF PANEL ESTIMATORS")
print("=" * 60)

comparison = pd.DataFrame({
    'Pooled_OLS': model_pooled.params.round(4),
    'Between': model_be.params.round(4),
})
# Add FE and FD (which don't include ed)
for var in ['exp', 'exp2', 'wks']:
    comparison.loc[var, 'FE'] = round(model_fe.params[var], 4)
    comparison.loc[var, 'FD'] = round(model_fd.params[var], 4)

for var in X.columns:
    comparison.loc[var, 'RE'] = round(model_re.params[var], 4)
```

```
print(comparison)

# 9. HAUSMAN TEST (FE vs RE)

print("\n" + "=" * 60)
print("HAUSMAN TEST: FE vs RE")
print("=" * 60)

# Manual Hausman test
# Compare coefficients from FE and RE on common variables
common_vars = ['exp', 'exp2', 'wks']
b_fe = np.array([model_fe.params[v] for v in common_vars])
b_re = np.array([model_re.params[v] for v in common_vars])

# Get variance matrices
model_fe_h = PanelOLS(y, df[common_vars], entity_effects=True).fit()
model_re_h = RandomEffects(y, X).fit()

V_fe = model_fe_h.cov.loc[common_vars, common_vars].values
V_re = model_re_h.cov.loc[common_vars, common_vars].values

diff = b_fe - b_re
V_diff = V_fe - V_re

try:
    hausman_stat = diff @ np.linalg.inv(V_diff) @ diff
    hausman_pval = 1 - stats.chi2.cdf(hausman_stat, len(common_vars))
    print(f"  Hausman statistic: {hausman_stat:.4f}")
    print(f"  Degrees of freedom: {len(common_vars)}")
    print(f"  p-value: {hausman_pval:.4f}")
    if hausman_pval < 0.05:
        print("  -> REJECT H0: Use Fixed Effects (alpha_i correlated with X)")
    else:
        print("  -> Fail to reject H0: Random Effects is consistent and efficient")
except np.linalg.LinAlgError:
    print("  Hausman test: V_FE - V_RE is not positive definite.")
    print("  Use the robust Hausman test (Wooldridge method) instead.")

# PART B: NONLINEAR PANEL MODELS (RAND HIE)

print("\n" + "=" * 60)
print("PART B: NONLINEAR PANEL MODELS (RAND HIE)")
print("=" * 60)

df_np = pd.read_excel("Chap6_npanel.xlsx")
print(f"\nDataset: RAND Health Insurance Experiment")
print(f"Total observations: {len(df_np)}")

np_vars = ['dmdu', 'med', 'mdu', 'lcoins', 'ndisease', 'female', 'age', 'lfam', 'child']
available = [v for v in np_vars if v in df_np.columns]
print(f"\n--- Summary Statistics ---")
print(df_np[available].describe().round(3))

# 10. BINARY OUTCOME: LOGIT MODELS
```

```
if 'dmdu' in df_np.columns:
    print("\n--- Binary Outcome: Pooled Logit ---")
    xvars = [v for v in ['lcoins', 'ndisease', 'female', 'age', 'lfam', 'child']
             if v in df_np.columns]
    df_logit = df_np[['dmdu'] + xvars + ['id']].dropna()

    X_logit = sm.add_constant(df_logit[xvars])
    y_logit = df_logit['dmdu']

    # Pooled logit with cluster-robust SEs
    model_logit = sm.Logit(y_logit, X_logit).fit(
        cov_type='cluster', cov_kwds={'groups': df_logit['id']}, disp=0)
    print(model_logit.summary2().tables[1].round(4))

    # Marginal effects
    print("\n--- Average Marginal Effects (Logit) ---")
    mfx = model_logit.get_margeff()
    print(mfx.summary())

# 11. COUNT OUTCOME: POISSON AND NEGATIVE BINOMIAL

if 'mdu' in df_np.columns:
    print("\n--- Count Outcome: Pooled Poisson ---")
    xvars_count = [v for v in ['lcoins', 'ndisease', 'female', 'age', 'lfam', 'child']
                   if v in df_np.columns]
    df_count = df_np[['mdu'] + xvars_count + ['id']].dropna()

    mean_mdu = df_count['mdu'].mean()
    var_mdu = df_count['mdu'].var()
    print(f"  Mean(mdu) = {mean_mdu:.3f}, Var(mdu) = {var_mdu:.3f}, Ratio =
{var_mdu/mean_mdu:.3f}")

    X_count = sm.add_constant(df_count[xvars_count])
    y_count = df_count['mdu']

    # Poisson with cluster-robust SEs
    model_pois = sm.GLM(y_count, X_count, family=sm.families.Poisson()).fit(
        cov_type='cluster', cov_kwds={'groups': df_count['id']})
    print(model_pois.summary2().tables[1].round(4))

    # Negative Binomial
    print("\n--- Count Outcome: Negative Binomial ---")
    model_nb = sm.GLM(y_count, X_count, family=sm.families.NegativeBinomial(alpha=1.0)).fit(
        cov_type='cluster', cov_kwds={'groups': df_count['id']})
    print(model_nb.summary2().tables[1].round(4))

    # Comparison
    print("\n--- Poisson vs Negative Binomial ---")
    comp_count = pd.DataFrame({
        'Poisson': model_pois.params.round(4),
        'NegBin': model_nb.params.round(4),
    })
    print(comp_count)

print("\n" + "=" * 60)
```

```
print("END OF CHAPTER 6")
print("=" * 60)
```

Appendix I: Python Code for Chapter 7 - Regression Discontinuity Designs

This appendix contains the complete Python code that replicates the Stata analysis from Chapter 7. The code is designed to run on Google Colab.

Setup Information

Dataset: Chap7_headstart.xlsx
Description: Head Start program data (Ludwig & Miller 2007)
Required Python packages: pandas, numpy, statsmodels, scipy, matplotlib, rdrobust (optional)

To run on Google Colab:
1. Upload the Excel file(s) to your Colab session
2. Install packages: !pip install pandas, numpy, statsmodels, scipy, matplotlib, rdrobust (optional)
3. Run the code cells below

Chapter 7 Python Code

```
"""
============================================================================
EE627 Microeconometrics - Chapter 7
Regression Discontinuity Designs
============================================================================
Instructor: Asst. Prof. Dr. Tiraphap Fakthong
Thammasat University

Dataset: Chap7_headstart.xlsx (Ludwig & Miller, 2007, QJE)
This script replicates "Chap 7 CT_2021_NBER.do" in Python.
Designed to run on Google Colab.

Required packages:
    pip install pandas numpy statsmodels scipy matplotlib rdrobust openpyxl
    (rdrobust: pip install rdrobust  -- Python version of the Stata package)
============================================================================
"""

# SETUP

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm

np.set_printoptions(precision=4)

print("=" * 60)
print("CHAPTER 7: REGRESSION DISCONTINUITY DESIGNS")
print("=" * 60)

# 1. LOAD DATA

df = pd.read_excel("Chap7_headstart.xlsx")
print(f"\nDataset: Head Start (Ludwig & Miller, 2007)")
```

```
print(f"Observations: {len(df)}")

# Define variables (matching Stata globals)
Y = 'mort_age59_related_postHS'   # Outcome: child mortality
X = 'povrate60'                    # Running variable: 1960 poverty rate
C = 59.1984                        # Cutoff

covariates = [c for c in ['census1960_pop', 'census1960_pctsch1417',
              'census1960_pctsch534', 'census1960_pctsch25plus',
              'census1960_pop1417', 'census1960_pop534',
              'census1960_pop25plus', 'census1960_pcturban',
              'census1960_pctblack'] if c in df.columns]

# Create running variable centered at cutoff
df['R'] = df[X] - C
df['T'] = (df[X] >= C).astype(int)

print(f"\nCutoff: {C}")
print(f"\n--- Summary Statistics ---")
print(df[[Y, X]].describe().round(3))
print(f"\nTreatment/Control split:")
print(df['T'].value_counts())

# 2. T-TEST: SIMPLE DIFFERENCE IN MEANS

print("\n" + "=" * 60)
print("2. SIMPLE DIFFERENCE IN MEANS")
print("=" * 60)

treated = df[df['T'] == 1][Y].dropna()
control = df[df['T'] == 0][Y].dropna()

t_stat, p_val = stats.ttest_ind(treated, control)
print(f"  Mean (treated):  {treated.mean():.4f}")
print(f"  Mean (control):  {control.mean():.4f}")
print(f"  Difference:      {treated.mean() - control.mean():.4f}")
print(f"  t-statistic:     {t_stat:.4f}")
print(f"  p-value:         {p_val:.4f}")

# 3. RD PLOT

print("\n" + "=" * 60)
print("3. RD PLOT")
print("=" * 60)

fig, ax = plt.subplots(figsize=(10, 6))

# Bin scatter plot
df_plot = df[[X, Y]].dropna()
n_bins = 40
df_plot['bin'] = pd.cut(df_plot[X], bins=n_bins)
bin_means = df_plot.groupby('bin')[Y].mean().reset_index()
bin_means['x_mid'] = bin_means['bin'].apply(lambda b: b.mid)

# Plot bins
below = bin_means[bin_means['x_mid'] < C]
above = bin_means[bin_means['x_mid'] >= C]
```

```
ax.scatter(below['x_mid'], below[Y], color='#4472C4', s=40, zorder=5, label='Control')
ax.scatter(above['x_mid'], above[Y], color='#ED7D31', s=40, zorder=5, label='Treated')

# Fit local polynomial on each side
for side, color, label_side in [(df_plot[df_plot[X] < C], '#4472C4', 'Below'),
                                (df_plot[df_plot[X] >= C], '#ED7D31', 'Above')]:
    if len(side) > 10:
        z = np.polyfit(side[X], side[Y], 2)
        p = np.poly1d(z)
        x_range = np.linspace(side[X].min(), side[X].max(), 100)
        ax.plot(x_range, p(x_range), color=color, linewidth=2)

ax.axvline(x=C, color='red', linestyle='--', linewidth=2, label=f'Cutoff ({C})')
ax.set_xlabel('Poverty Rate 1960', fontsize=12)
ax.set_ylabel('Child Mortality (age 5-9)', fontsize=12)
ax.set_title('RD Plot: Head Start Program Effect on Child Mortality', fontweight='bold',
fontsize=13)
ax.legend()
plt.tight_layout()
plt.savefig('fig_ch7_rdplot.png', dpi=150)
plt.show()
print("Figure saved: fig_ch7_rdplot.png")

# 4. PARAMETRIC RD (Replicating Ludwig & Miller)

print("\n" + "=" * 60)
print("4. PARAMETRIC RD (Local Linear, h=9)")
print("=" * 60)

# Stata: reg $Y T##c.R if abs(R)<=9, robust
h = 9  # bandwidth
df_rd = df[abs(df['R']) <= h].copy()

# Create interaction term
df_rd['T_x_R'] = df_rd['T'] * df_rd['R']
X_rd = sm.add_constant(df_rd[['T', 'R', 'T_x_R']])
y_rd = df_rd[Y]

model_rd = sm.OLS(y_rd, X_rd).fit(cov_type='HC1')
print(model_rd.summary2().tables[1].round(4))
print(f"\n  RD Estimate (coefficient on T): {model_rd.params['T']:.4f}")
print(f"  Robust SE: {model_rd.bse['T']:.4f}")
print(f"  p-value: {model_rd.pvalues['T']:.4f}")

# 5. RDROBUST (if available)

print("\n" + "=" * 60)
print("5. RDROBUST ESTIMATION")
print("=" * 60)

try:
    from rdrobust import rdrobust, rdplot as rd_plot_func, rdbwselect

    print("\n--- rdrobust: Default (MSE-optimal bandwidth, triangular kernel) ---")
    rd_result = rdrobust(df[Y].values, df[X].values, c=C)
    print(f"  Point estimate:  {rd_result.Estimate.iloc[0]:.4f}")
    print(f"  Robust p-value:  {rd_result.pv.iloc[2]:.4f}")
```

```
    print(f"  Bandwidth (h):   {rd_result.bws.iloc[0, 0]:.4f}")
    print(rd_result)

    print("\n--- rdrobust: Uniform kernel, h=9 (replicate Ludwig & Miller) ---")
    rd_result2 = rdrobust(df[Y].values, df[X].values, c=C, h=9, kernel='uniform')
    print(f"  Point estimate:  {rd_result2.Estimate.iloc[0]:.4f}")
    print(f"  Robust p-value:  {rd_result2.pv.iloc[2]:.4f}")

    print("\n--- Bandwidth selection ---")
    bw_result = rdbwselect(df[Y].values, df[X].values, c=C, kernel='uniform', all=True)
    print(bw_result)

except ImportError:
    print("  rdrobust package not installed.")
    print("  Install with: pip install rdrobust")
    print("  Using manual parametric RD instead (see section 4 above).")

# 6. RD WITH DIFFERENT BANDWIDTHS

print("\n" + "=" * 60)
print("6. SENSITIVITY TO BANDWIDTH CHOICE")
print("=" * 60)

bandwidths = [4, 6, 8, 9, 10, 12, 15]
print(f"\n  {'Bandwidth':>10s}  {'RD Estimate':>12s}  {'SE':>8s}  {'p-value':>8s}
{'N':>6s}")
print(f"  {'-'*50}")

for h_val in bandwidths:
    df_h = df[abs(df['R']) <= h_val].copy()
    df_h['T_x_R'] = df_h['T'] * df_h['R']
    X_h = sm.add_constant(df_h[['T', 'R', 'T_x_R']])
    y_h = df_h[Y]
    model_h = sm.OLS(y_h, X_h).fit(cov_type='HC1')
    print(f"  {h_val:>10d}  {model_h.params['T']:>12.4f}  "
          f"{model_h.bse['T']:>8.4f}  {model_h.pvalues['T']:>8.4f}  {len(df_h):>6d}")

# 7. FALSIFICATION TESTS

print("\n" + "=" * 60)
print("7. FALSIFICATION TESTS")
print("=" * 60)

# --- Density test (McCrary-style) ---
print("\n--- A. Density Test (Histogram) ---")
fig, ax = plt.subplots(figsize=(10, 5))
bins_left = np.arange(df[X].min(), C, 2)
bins_right = np.arange(C, df[X].max() + 2, 2)
ax.hist(df[df[X] < C][X].dropna(), bins=bins_left, alpha=0.7, color='#4472C4',
edgecolor='white', label='Control')
ax.hist(df[df[X] >= C][X].dropna(), bins=bins_right, alpha=0.7, color='#ED7D31',
edgecolor='white', label='Treated')
ax.axvline(x=C, color='red', linestyle='--', linewidth=2)
ax.set_xlabel('Poverty Rate 1960')
ax.set_ylabel('Frequency')
ax.set_title('Density of Running Variable at Cutoff', fontweight='bold')
ax.legend()
```

```
plt.tight_layout()
plt.savefig('fig_ch7_density.png', dpi=150)
plt.show()

# --- Covariate balance test ---
print("\n--- B. Covariate Balance Test ---")
h_test = 9
df_test = df[abs(df['R']) <= h_test].copy()

if covariates:
    print(f"  Testing pre-treatment covariates (h={h_test}):")
    for cov in covariates[:5]:
        df_cov = df_test[[cov, 'T', 'R']].dropna()
        df_cov['T_x_R'] = df_cov['T'] * df_cov['R']
        X_cov = sm.add_constant(df_cov[['T', 'R', 'T_x_R']])
        model_cov = sm.OLS(df_cov[cov], X_cov).fit(cov_type='HC1')
        print(f"    {cov:30s}: coef(T)={model_cov.params['T']:>10.4f}, "
              f"p={model_cov.pvalues['T']:.4f} {'*' if model_cov.pvalues['T'] < 0.05 else
''}")

# --- Placebo outcome ---
if 'mort_age25plus_related_postHS' in df.columns:
    print("\n--- C. Placebo Outcome Test ---")
    placebo_var = 'mort_age25plus_related_postHS'
    df_plac = df_test[[placebo_var, 'T', 'R']].dropna()
    df_plac['T_x_R'] = df_plac['T'] * df_plac['R']
    X_plac = sm.add_constant(df_plac[['T', 'R', 'T_x_R']])
    model_plac = sm.OLS(df_plac[placebo_var], X_plac).fit(cov_type='HC1')
    print(f"    Placebo outcome ({placebo_var}):")
    print(f"    RD estimate: {model_plac.params['T']:.4f}, p={model_plac.pvalues['T']:.4f}")

# --- Placebo cutoff ---
print("\n--- D. Placebo Cutoff Test ---")
for fake_c in [2, 3, -2, -3]:
    df_fake = df[df['R'] > 0].copy() if fake_c > 0 else df[df['R'] < 0].copy()
    df_fake['T_fake'] = (df_fake['R'] >= fake_c).astype(int)
    df_fake['R_centered'] = df_fake['R'] - fake_c
    df_fake['T_x_R'] = df_fake['T_fake'] * df_fake['R_centered']
    df_fake2 = df_fake[[Y, 'T_fake', 'R_centered', 'T_x_R']].dropna()
    if len(df_fake2) > 20:
        X_fake = sm.add_constant(df_fake2[['T_fake', 'R_centered', 'T_x_R']])
        model_fake = sm.OLS(df_fake2[Y], X_fake).fit(cov_type='HC1')
        print(f"    Cutoff at R={fake_c:+d}: estimate={model_fake.params['T_fake']:>8.4f}, "
              f"p={model_fake.pvalues['T_fake']:.4f}")


print("\n" + "=" * 60)
print("END OF CHAPTER 7")
print("=" * 60)
```

Appendix J: Python Code for Chapter 8 - Difference-in-Differences

This appendix contains the complete Python code that replicates the Stata analysis from Chapter 8. The code is designed to run on Google Colab.

Setup Information

Dataset: Chap8_autor.xlsx
Description: Employment-at-will data (Autor 2003)
Required Python packages: pandas, numpy, statsmodels, scipy, matplotlib

To run on Google Colab:
1. Upload the Excel file(s) to your Colab session
2. Install packages: !pip install pandas, numpy, statsmodels, scipy, matplotlib
3. Run the code cells below

Chapter 8 Python Code

```
"""
============================================================================
EE627 Microeconometrics - Chapter 8
Difference-in-Differences
============================================================================
Instructor: Asst. Prof. Dr. Tiraphap Fakthong
Thammasat University

Dataset: Chap8_autor.xlsx (Autor, 2003, JOLE)
This script replicates "Chap 8 DID_2024.do" in Python.
Designed to run on Google Colab.

Required packages:
    pip install pandas numpy statsmodels scipy matplotlib linearmodels openpyxl
============================================================================
"""

# SETUP

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf

np.set_printoptions(precision=4)
pd.set_option('display.max_columns', 15)

print("=" * 60)
print("CHAPTER 8: DIFFERENCE-IN-DIFFERENCES")
print("=" * 60)

# 1. LOAD DATA
```

```
df = pd.read_excel("Chap8_autor.xlsx")
print(f"\nDataset: Autor (2003, JOLE) - Employment-at-Will Exceptions")
print(f"Total observations: {len(df)}")

# Check available variables
needed_vars = ['state', 'year', 'lstateths', 'lnemp', 'mico', 'mppa', 'mgfa']
avail = [v for v in needed_vars if v in df.columns]
print(f"Available variables: {avail}")

# Sort and restrict sample
df = df.sort_values(['state', 'year'])
df = df[(df['year'] >= 79) & (df['year'] <= 95)].copy()
print(f"Sample after restricting to 1979-1995: {len(df)}")

print(f"\n--- Summary Statistics ---")
print(df[['lstateths', 'lnemp', 'mico', 'mppa', 'mgfa']].describe().round(3))

# PART 1: DESCRIPTIVE ANALYSIS

print("\n" + "=" * 60)
print("PART 1: DESCRIPTIVE ANALYSIS")
print("=" * 60)

# Step 1.3: Regress lstateths on lnemp, keep residuals
model_partial = sm.OLS(
    df['lstateths'],
    sm.add_constant(df['lnemp'])
).fit()
df['ehat'] = model_partial.resid

# Step 1.4: Create relative growth (relative to 1979)
df = df.sort_values(['state', 'year'])
first_year_resid = df.groupby('state')['ehat'].first().reset_index()
first_year_resid.columns = ['state', 'a']
df = df.merge(first_year_resid, on='state')
df['lrelativeg'] = df['ehat'] - df['a']
df = df.drop(columns=['a', 'ehat'])

# Step 1.5: Create "any exception" indicator
df['any'] = ((df['mico'] + df['mppa'] + df['mgfa'] > 0) &
             (df['mico'] + df['mppa'] + df['mgfa'] < np.inf)).astype(int)

# Step 1.6: Collapse to year-level
year_data = df.groupby('year').agg(
    lrelativeg=('lrelativeg', 'mean'),
    any_count=('any', 'sum')
).reset_index()
year_data['year_full'] = year_data['year'] + 1900

# Step 1.7: Two-axis plot
fig, ax1 = plt.subplots(figsize=(10, 6))
ax2 = ax1.twinx()

ax1.plot(year_data['year_full'], year_data['lrelativeg'], 'D-',
         color='#4472C4', linewidth=2, markersize=6, label='Log relative growth of THS
employment')
```

```
ax1.set_xlabel('Year', fontsize=12)
ax1.set_ylabel('Log THS Industry Size Relative to 1979', fontsize=11, color='#4472C4')
ax1.axhline(y=0, color='black', linewidth=0.5)

ax2.plot(year_data['year_full'], year_data['any_count'], '^-',
         color='#ED7D31', linewidth=2, markersize=6, label='States recognizing an
exception')
ax2.set_ylabel('Number of States Recognizing An Exception', fontsize=11, color='#ED7D31')

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left', fontsize=9)

plt.title('Relative THS Growth vs Employment-at-Will Exceptions', fontweight='bold',
fontsize=13)
plt.tight_layout()
plt.savefig('fig_ch8_did_descriptive.png', dpi=150)
plt.show()
print("Figure saved: fig_ch8_did_descriptive.png")

# PART 2: DID REGRESSION (Table 3 Replication)

print("\n" + "=" * 60)
print("PART 2: DID REGRESSION (Table 3 Replication)")
print("=" * 60)

# Reload original data for regression
df2 = pd.read_excel("Chap8_autor.xlsx")
df2 = df2.sort_values(['state', 'year'])

# Create time trend
df2['t'] = df2['year'] - 78

# Create dummies for state and year fixed effects
state_dummies = pd.get_dummies(df2['state'], prefix='state', drop_first=True, dtype=float)
year_dummies = pd.get_dummies(df2['year'], prefix='year', drop_first=True, dtype=float)

# Create state-specific trends
state_dummies_trend = pd.get_dummies(df2['state'], prefix='st', dtype=float)
state_trends = state_dummies_trend.multiply(df2['t'].values, axis=0)
state_trends.columns = [col + '_t' for col in state_trends.columns]
state_trends = state_trends.iloc[:, 1:]  # drop first for identification

y = df2['lstateths']
base_X = pd.concat([state_dummies, year_dummies], axis=1)
base_X_trend = pd.concat([state_dummies, year_dummies, state_trends], axis=1)

results = {}

# Function to run DID regression with cluster-robust SEs
def run_did(y, treatment_vars, X_base, cluster_var, label):
    """Run DID regression with state and year FE, clustered SEs"""
    X = X_base.copy()
    for tv in treatment_vars:
        X[tv] = df2[tv].values
    X = sm.add_constant(X)

    # Drop rows with NaN
    mask = y.notna() & X.notna().all(axis=1)
```

```
    y_clean = y[mask]
    X_clean = X[mask]
    cluster_clean = cluster_var[mask]

    model = sm.OLS(y_clean, X_clean).fit(
        cov_type='cluster', cov_kwds={'groups': cluster_clean})

    result = {}
    for tv in treatment_vars:
        result[tv] = {
            'coef': model.params[tv],
            'se': model.bse[tv],
            'pval': model.pvalues[tv]
        }
    result['r2'] = model.rsquared
    result['n'] = model.nobs
    return result

cluster = df2['state']

# Column 1: mico only, no state trends
print("\n--- Running 8 specifications (Table 3) ---")
specs = [

    ('Col 1', ['mico'], base_X, 'State+Year FE'),
    ('Col 2', ['mico'], base_X_trend, 'State+Year FE + State trends'),
    ('Col 3', ['mppa'], base_X, 'State+Year FE'),
    ('Col 4', ['mppa'], base_X_trend, 'State+Year FE + State trends'),
    ('Col 5', ['mgfa'], base_X, 'State+Year FE'),
    ('Col 6', ['mgfa'], base_X_trend, 'State+Year FE + State trends'),
    ('Col 7', ['mico', 'mppa', 'mgfa'], base_X, 'State+Year FE'),
    ('Col 8', ['mico', 'mppa', 'mgfa'], base_X_trend, 'State+Year FE + State trends'),
]

all_results = {}
for label, tvars, xbase, desc in specs:
    try:
        res = run_did(y, tvars, xbase, cluster, label)
        all_results[label] = res
        print(f"  {label} ({desc}):")
        for tv in tvars:
            star = '***' if res[tv]['pval'] < 0.01 else '**' if res[tv]['pval'] < 0.05 else
'*' if res[tv]['pval'] < 0.10 else ''
            print(f"    {tv}: {res[tv]['coef']:.4f} ({res[tv]['se']:.4f}){star}")
        print(f"    R2={res['r2']:.4f}, N={res['n']:.0f}")
    except Exception as e:
        print(f"  {label}: Error - {e}")

# 3. RESULTS TABLE

print("\n" + "=" * 60)
print("SUMMARY TABLE: DID ESTIMATES")
print("=" * 60)

print(f"\n  {'':8s} {'Col1':>8s} {'Col2':>8s} {'Col3':>8s} {'Col4':>8s} "
      f"{'Col5':>8s} {'Col6':>8s} {'Col7':>8s} {'Col8':>8s}")
print(f"  {'-'*72}")
```

```
for var in ['mico', 'mppa', 'mgfa']:
    row = f"  {var:8s}"
    for col_label in ['Col 1', 'Col 2', 'Col 3', 'Col 4', 'Col 5', 'Col 6', 'Col 7', 'Col
8']:
        if col_label in all_results and var in all_results[col_label]:
            coef = all_results[col_label][var]['coef']
            pval = all_results[col_label][var]['pval']
            star = '***' if pval < 0.01 else '**' if pval < 0.05 else '*' if pval < 0.10
else ''
            row += f" {coef:>7.3f}{star}"
        else:
            row += f" {'--':>8s}"
    print(row)

print(f"\n  {'Trends':8s}", end="")
for i in range(8):
    print(f" {'Yes' if i % 2 == 1 else 'No':>8s}", end="")
print()

# 4. INTERPRETATION

print("\n" + "=" * 60)
print("4. INTERPRETATION OF DID RESULTS")
print("=" * 60)

print("""
Key findings from Autor (2003):

1. IMPLIED CONTRACT (mico):
   - Positive and statistically significant in BOTH specifications
   - Robust to inclusion of state-specific time trends
   - Interpretation: Adoption of implied contract exceptions INCREASES
     temporary help services (THS) employment
   - Mechanism: Workers seek temp agency employment for protection
     against wrongful discharge

2. PUBLIC POLICY (mppa) and GOOD FAITH (mgfa):
   - Results change substantially when state trends are included
   - NOT robust -- suggests confounding state-level trends
   - The parallel trends assumption may not hold for these exceptions

3. WHY ROBUSTNESS TO STATE TRENDS MATTERS:
   - DID relies on the parallel trends assumption
   - If states that adopt exceptions were already trending differently,
     the DID estimate is biased
   - Including state x time trends allows for differential trends
   - If the estimate survives, the causal interpretation is stronger
""")

print("\n" + "=" * 60)
print("END OF CHAPTER 8")
print("=" * 60)
```

References

Primary Textbooks

Angrist, J. D., & Pischke, J.-S. (2009). Mostly harmless econometrics: An empiricist's companion. Princeton University Press.

Cameron, A. C., & Trivedi, P. K. (2005). Microeconometrics: Methods and applications. Cambridge University Press.

Cameron, A. C., & Trivedi, P. K. (2009). Microeconometrics using Stata. Stata Press.

Cameron, A. C., & Trivedi, P. K. (2010). Microeconometrics using Stata (Rev. ed.). Stata Press.

Greene, W. H. (2018). Econometric analysis (8th ed.). Pearson.

Wooldridge, J. M. (2002). Econometric analysis of cross section and panel data. MIT Press.

Wooldridge, J. M. (2010). Econometric analysis of cross section and panel data (2nd ed.). MIT Press.

Wooldridge, J. M. (2020). Introductory econometrics: A modern approach (7th ed.). Cengage Learning.

Chapter 1: OLS Regression and Prediction

Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. Journal of the Royal Statistical Society: Series B (Methodological), 26(2), 211–252. https://doi.org/10.1111/j.2517-6161.1964.tb00553.x

Breusch, T. S., & Pagan, A. R. (1979). A simple test for heteroscedasticity and random coefficient variation. Econometrica, 47(5), 1287–1294. https://doi.org/10.2307/1911963

Duan, N. (1983). Smearing estimate: A nonparametric retransformation method. Journal of the American Statistical Association, 78(383), 605–610. https://doi.org/10.1080/01621459.1983.10478017

Engel, E. (1857). Die Productions- und Consumtionsverhältnisse des Königreichs Sachsen. Zeitschrift des Statistischen Bureaus des Königlich Sächsischen Ministeriums des Innern, 8–9, 1–54.

Huber, P. J. (1967). The behavior of maximum likelihood estimates under nonstandard conditions. In L. M. Le Cam & J. Neyman (Eds.), Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability (Vol. 1, pp. 221–233). University of California Press.

Marschak, J., & Andrews, W. H. (1944). Random simultaneous equations and the theory of production. Econometrica, 12(3/4), 143–205. https://doi.org/10.2307/1905432

Ramsey, J. B. (1969). Tests for specification errors in classical linear least-squares regression analysis. Journal of the Royal Statistical Society: Series B (Methodological), 31(2), 350–371. https://doi.org/10.1111/j.2517-6161.1969.tb00796.x

White, H. (1980). A heteroskedasticity-consistent covariance matrix estimator and a direct test for heteroskedasticity. Econometrica, 48(4), 817–838. https://doi.org/10.2307/1912934

Chapter 2: Monte Carlo Simulation

Lindeberg, J. W. (1922). Eine neue Herleitung des Exponentialgesetzes in der Wahrscheinlichkeitsrechnung. Mathematische Zeitschrift, 15(1), 211–225. https://doi.org/10.1007/BF01494395

Lévy, P. (1925). Calcul des probabilités. Gauthier-Villars.

Chapter 3: Heteroskedasticity, SUR, and Survey Data

Aitken, A. C. (1936). On least squares and linear combination of observations. Proceedings of the Royal Society of Edinburgh, 55, 42–48. https://doi.org/10.1017/S0370164600014346

Kish, L. (1965). Survey sampling. John Wiley & Sons.

Zellner, A. (1962). An efficient method of estimating seemingly unrelated regressions and tests for aggregation bias. Journal of the American Statistical Association, 57(298), 348–368. https://doi.org/10.1080/01621459.1962.10480664

Chapter 4: Instrumental Variables

Baum, C. F., Schaffer, M. E., & Stillman, S. (2007). Enhanced routines for instrumental variables/generalized method of moments estimation and testing. The Stata Journal, 7(4), 465–506. https://doi.org/10.1177/1536867X0800700402

Durbin, J. (1954). Errors in variables. Review of the International Statistical Institute, 22(1/3), 23–32. https://doi.org/10.2307/1401917

Hansen, L. P. (1982). Large sample properties of generalized method of moments estimators. Econometrica, 50(4), 1029–1054. https://doi.org/10.2307/1912775

Hausman, J. A. (1978). Specification tests in econometrics. Econometrica, 46(6), 1251–1271. https://doi.org/10.2307/1913827

Sargan, J. D. (1958). The estimation of economic relationships using instrumental variables. Econometrica, 26(3), 393–415. https://doi.org/10.2307/1907619

Staiger, D., & Stock, J. H. (1997). Instrumental variables regression with weak instruments. Econometrica, 65(3), 557–586. https://doi.org/10.2307/2171753

Stock, J. H., & Yogo, M. (2005). Testing for weak instruments in linear IV regression. In D. W. K. Andrews & J. H. Stock (Eds.), Identification and inference for econometric models: Essays in honor of Thomas Rothenberg (pp. 80–108). Cambridge University Press.

Wu, D.-M. (1973). Alternative tests of independence between stochastic regressors and disturbances. Econometrica, 41(4), 733–750. https://doi.org/10.2307/1914093

Chapter 5: Quantile Regression and Count Data

Koenker, R., & Bassett, G. (1978). Regression quantiles. Econometrica, 46(1), 33–50. https://doi.org/10.2307/1913643

Koenker, R. (2005). Quantile regression. Cambridge University Press.

Machado, J. A. F., & Santos Silva, J. M. C. (2005). Quantiles for counts. Journal of the American Statistical Association, 100(472), 1226–1237. https://doi.org/10.1198/016214505000000330

Chapter 6: Panel Data

Baltagi, B. H. (2021). Econometric analysis of panel data (6th ed.). Springer.

Baltagi, B. H., & Khanti-Akom, S. (1990). On efficient estimation with panel data: An empirical comparison of instrumental variables estimators. Journal of Applied Econometrics, 5(4), 401–406. https://doi.org/10.1002/jae.3950050408

Chamberlain, G. (1980). Analysis of covariance with qualitative data. The Review of Economic Studies, 47(1), 225–238. https://doi.org/10.2307/2297110

Cornwell, C., & Rupert, P. (1988). Efficient estimation with panel data: An empirical comparison of instrumental variables estimators. Journal of Applied Econometrics, 3(2), 149–155. https://doi.org/10.1002/jae.3950030206

Deb, P., & Trivedi, P. K. (2002). The structure of demand for health care: Latent class versus two-part models. Journal of Health Economics, 21(4), 601–625. https://doi.org/10.1016/S0167-6296(02)00008-5

Manning, W. G., Newhouse, J. P., Duan, N., Keeler, E. B., & Leibowitz, A. (1987). Health insurance and the demand for medical care: Evidence from a randomized experiment. The American Economic Review, 77(3), 251–277.

Mundlak, Y. (1978). On the pooling of time series and cross section data. Econometrica, 46(1), 69–85. https://doi.org/10.2307/1913646

Chapter 7: Regression Discontinuity

Calonico, S., Cattaneo, M. D., & Titiunik, R. (2014). Robust nonparametric confidence intervals for regression-discontinuity designs. Econometrica, 82(6), 2295–2326. https://doi.org/10.3982/ECTA11757

Cattaneo, M. D., Idrobo, N., & Titiunik, R. (2020). A practical introduction to regression discontinuity designs: Foundations. Cambridge University Press.

Cattaneo, M. D., Keele, L., & Titiunik, R. (2023). A guide to regression discontinuity designs in medical applications. Statistics in Medicine, 42(24), 4484–4513. https://doi.org/10.1002/sim.9861

Cattaneo, M. D., & Titiunik, R. (2022). Regression discontinuity designs. Annual Review of Economics, 14, 821–851. https://doi.org/10.1146/annurev-economics-051520-021409

Hahn, J., Todd, P., & Van der Klaauw, W. (2001). Identification and estimation of treatment effects with a regression-discontinuity design. Econometrica, 69(1), 201–209. https://www.jstor.org/stable/2692190

Imbens, G. W., & Lemieux, T. (2008). Regression discontinuity designs: A guide to practice. Journal of Econometrics, 142(2), 615–635. https://doi.org/10.1016/j.jeconom.2007.05.001

Lee, D. S., & Lemieux, T. (2010). Regression discontinuity designs in economics. Journal of Economic Literature, 48(2), 281–355. https://doi.org/10.1257/jel.48.2.281

Ludwig, J., & Miller, D. L. (2007). Does Head Start improve children's life chances? Evidence from a regression discontinuity design. The Quarterly Journal of Economics, 122(1), 159–208. https://doi.org/10.1162/qjec.122.1.159

McCrary, J. (2008). Manipulation of the running variable in the regression discontinuity design: A density test. Journal of Econometrics, 142(2), 698–714. https://doi.org/10.1016/j.jeconom.2007.05.005

Chapter 8: Difference-in-Differences

Autor, D. H. (2003). Outsourcing at will: The contribution of unjust dismissal doctrine to the growth of employment outsourcing. Journal of Labor Economics, 21(1), 1–42. https://doi.org/10.1086/344122

Bertrand, M., Duflo, E., & Mullainathan, S. (2004). How much should we trust differences-in-differences estimates? The Quarterly Journal of Economics, 119(1), 249–275. https://doi.org/10.1162/003355304772839588

Holland, P. W. (1986). Statistics and causal inference. Journal of the American Statistical Association, 81(396), 945–960. https://doi.org/10.1080/01621459.1986.10478354

Rubin, D. B. (1974). Estimating causal effects of treatments in randomized and nonrandomized studies. Journal of Educational Psychology, 66(5), 688–701. https://doi.org/10.1037/h0037350

Rubin, D. B. (1986). Comment: Which ifs have causal answers. Journal of the American Statistical Association, 81(396), 961–962. https://doi.org/10.1080/01621459.1986.10478355

Statistical Software and Packages

Cattaneo, M. D., Idrobo, N., & Titiunik, R. (2021). rdrobust, rdplot, rdbwselect, rddensity, rdlocrand: Stata and R/Python packages for regression discontinuity designs [Computer software]. https://rdpackages.github.io/

StataCorp. (2023). Stata statistical software: Release 18 [Computer software]. StataCorp LLC.